MULTI-ENVIRONMENT CONFIGURATION MANAGEMENT WITH HELM

A PROJECT REPORT

Submitted by

Arun Kumar T (Reg.No.953724243007)

Priyan M (Reg.No.953724243040)

In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

AAA COLLEGE OF ENGINEERING AND TECHNOLOGY, SIVAKASI

(AN AUTONOMOUS INSTITUTION)

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**Multi-Environment Configuration Management with Helm**" is the bonafide work of **Arun Kumar T (Reg.No.953724243007), Priyan M (Reg. No.953724243040)**, who carried out this project under my supervision.

SIGNATURE

SUPERVISOR NAME, QUALIFICATION
SUPERVISOR &
DESIGNATION
Department of Artificial Intelligence and Data Science
AAA College of Engineering and Technology
Sivakasi

SIGNATURE

Dr.M.KARUPPASAMY, M.Tech., Ph.D.,
ASSOCIATE PROFESSOR &
HEAD OF THE DEPARTMENT
Department of Artificial Intelligence and Data Science
AAA College of Engineering and Technology
Sivakasi

Submitted for viva - voce examination held on …….

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We would first like to thank Almighty for the blessings that made this work possible. We thank our Management, AAA College of Engineering and Technology, Sivakasi, for providing the facilities and infrastructure needed for the project.

We express our sincere thanks to **Dr. M. SEKAR M.E., Ph.D., (S.Korea)**, Principal, AAA College of Engineering and Technology, Sivakasi, for the necessary facilities and for permitting us to carry out this work.

We express our gratitude to **Dr.M.KARUPPASAMY M.Tech., Ph.D.**, Associate Professor and Head of the Department, Department of Artificial Intelligence and Data Science, for his constant encouragement, which helped us to complete the project.

We wish to express our deep sense of gratitude to our Project Guide **SUPERVISOR NAME, DESIGNATION**, for the guidance, supervision and information provided throughout the project.

We express our gratitude to our Project Coordinator **PROJECT COORDINATOR NAME, DESIGNATION**, Department of Artificial Intelligence and Data Science, for valuable advice and constant encouragement.

It is our privilege to record our gratitude to all the faculty members and supporting staff of the Department of Artificial Intelligence and Data Science, whose encouragement and suggestions have been a source of inspiration to us.

ABSTRACT

Teams that deploy one application to development, test, staging and production often copy a set of Kubernetes manifests for every environment. The copies drift apart, a change must be repeated several times, and a wrong value is found only after the deployment. This project describes the application once, as the Helm chart `multi-env-app`, and expresses every difference between the environments in a values file of 28 to 43 lines. A JSON Schema rejects unknown environments, wrong types, out-of-range numbers and secrets in the configuration before anything reaches a cluster, and a checksum annotation restarts the Pods when a setting changes.

The application is a Flask dashboard with Bootstrap that shows its name, environment, version and configuration. It is packaged as one image of 126 MB that is promoted unchanged through the four environments. Nine scripts install, validate, test, upgrade and roll back the releases, 62 pytest cases check the application and the chart, and a GitHub Actions workflow lints, renders and validates the chart, builds the image and deploys all four environments to a kind cluster.

On the cluster, all four releases were deployed in 40 s, 56 of 56 compared settings of the running Pods equalled the values files, all 26 live checks passed, a rollback restored an earlier revision, and `--atomic` restored a release after an upgrade to a non-existent image. A measurement of 13 rolling updates showed that `maxUnavailable: 0` keeps full capacity but still lost 1.2 to 1.5 % of the requests in short bursts, which a `preStop` delay removed; “zero downtime” is therefore not claimed. The report states for every result whether it was run on a cluster, run locally or derived. Minikube, which the scripts support, was not executed.

Keywords: Helm, Kubernetes, Configuration Management, Multi-Environment Deployment, Values Files, JSON Schema, Rolling Update, GitHub Actions, Release Rollback

TABLE OF CONTENTS

CHAPTER NO.	DESCRIPTION	PAGE NO.
	ABSTRACT	**iv**
	LIST OF FIGURES	**vii**
	LIST OF TABLES	**viii**
	LIST OF ABBREVIATIONS	**ix**
	GLOSSARY OF TECHNICAL TERMS	**ix**
I	**INTRODUCTION**	**1**
	1.1 Background and Motivation	1
	1.2 The Multi-Environment Configuration Problem	1
	1.3 Problem Statement	2
	1.4 Objectives of the Project	2
	1.5 Scope, Limitations and Contribution	3
	1.6 Organisation of the Report	3
II	**LITERATURE SURVEY**	**4**
	2.1 Research Papers and Documentation	4
	2.2 Existing Approaches to Configuration Management	4
	2.3 Research Gaps and Objective	5
III	**PROPOSED METHODOLOGY**	**6**
	3.1 Existing and Proposed System	6
	3.2 System Requirements	6
	3.3 Block Diagram and Explanation	7
	3.4 Application Design	8
	3.4.1 Routes, Configuration and Secret Handling	8
	3.4.2 Dashboard and Probes	9
	3.5 Containerisation	10
	3.6 Helm Chart Design	10
	3.7 Values and Environment-Specific Configuration	11
	3.7.1 Layering of Values	11
	3.7.2 Configuration of the Four Environments	12
	3.7.3 Schema Validation of Values	12
	3.8 Helm Templates	13
	3.8.1 Rolling-Update Strategy	14
	3.9 Kubernetes Topology and Objects	14
	3.10 Release Management with Helm	15
	3.11 Automation Scripts and Testing Strategy	16
	3.12 Continuous Integration with GitHub Actions	17
	3.13 Security and Secret Handling	17
	3.14 Resource Requirements of the Four Environments	18
IV	**RESULTS AND DISCUSSION**	**19**
	4.1 Basis of the Results	19
	4.2 Deployment of the Four Environments	19
	4.3 Dashboards of the Four Environments	21
	4.4 Test Cases and Results	23
	4.5 Reduction of Duplicated Configuration	25
	4.6 Schema Validation Results	26
	4.7 Upgrade and Rollback	27
	4.8 Forced Failures	28
	4.9 Availability during Rolling Updates	29

	4.10	Image and Pipeline Results	31
	4.11	Discussion	32
V	**CONCLUSION AND FUTURE ENHANCEMENTS**		**34**
	5.1	Conclusion	34
	5.2	Summary of Contributions	34
	5.3	Comparison with the Literature	35
	5.4	Practical Considerations and Limitations	35
	5.5	Future Enhancements	36
VI	**APPENDICES**		**37**
		Appendix A – Directory structure of the project	37
		Appendix B – Application source code	37
		Appendix C – Container files	42
		Appendix D – Helm chart	42
		Appendix E – Scripts	48
		Appendix F – GitHub Actions validation workflow	51
		Appendix G – Other files and reproduction of the results	53
VII	**REFERENCES**		**54**

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Copied manifests per environment compared with one chart and four values files	2
3.1	Block diagram of the multi-environment Helm solution	7
3.2	Application inside a Pod: sources of configuration, request flow and probes	8
3.3	Parts of the dashboard (production release)	10
3.4	Layered structure of the application image (instruction and purpose)	10
3.5	Values layering and schema validation	11
3.6	Cluster topology: one namespace and one Helm release per environment	15
3.7	Lifecycle of a Helm release	15
4.1	Helm releases after `scripts/deploy.sh all` (`helm list -A`, update time omitted)	19
4.2	Secrets of the production namespace (`kubectl get secret -n prod`)	20
4.3	Deployment of the production environment (`kubectl describe deployment`, selected lines)	20
4.4	Upper part of the dashboards of development, testing, staging and production (level A)	21
4.5	Dashboard of the production environment, served by the Pod myapp-prod-multi-env-app-78c6bddb45-4xx97	21
4.6	Logs of a development Pod and a production Pod (`kubectl logs`, probe requests omitted)	22
4.7	Result of the pytest run on the development machine	23
4.8	Output of `scripts/validate-helm.sh` (the three stages of each environment)	24
4.9	Lines to maintain: copied manifests compared with the chart and values files (level C)	25
4.10	Upgrade, history, rollback and history of the development release (update times omitted)	27
4.11	Dashboard of the development release after the upgrade (revision 2) and after the rollback (revision 3); upper part of the page	27
4.12	Failed upgrade with `--atomic`: the old Pod keeps running and Helm restores revision 1 as revision 3	28
4.13	Production installed without its Secret (`helm install`, `kubectl get pods`, events)	29
4.14	Failed requests per rollout in the five scenarios (bars: mean; dots: individual runs)	30
4.15	Timeline of the four jobs of the validation workflow (run 6, commit 9f0e6bc)	32
5.1	Roadmap of future enhancements	36

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	Project objectives and where they are addressed	2
2.1	Surveyed literature and its consequence for the design	4
2.2	Comparison of configuration-management approaches	5
3.1	Existing and proposed systems compared	6
3.2	Software and hardware environment	7
3.3	Application routes	9
3.4	Environment variables read by the application	9
3.5	Effective values of a staging release with two command-line overrides	11
3.6	Environment-specific configuration (values that differ)	12
3.7	Rules of values.schema.json	12
3.8	Rendered manifests and size of the environment files	14
3.9	Helm commands used in the project	16
3.10	Scripts of the project	16
3.11	Test levels	17
3.12	Jobs of the workflow helm-validation.yml	17
3.13	Security measures	18
3.14	Resources requested by the four environments (m = millicores, Mi = mebibytes)	18
4.1	Evidence levels used in this report	19
4.2	Releases of the four environments in the cluster (level A)	20
4.3	Settings reported by the running Pods through /api/config (level A)	22
4.4	Test cases and results	23
4.5	Live checks of the four environments (level A; – = check not made)	24
4.6	Invalid values rejected by values.schema.json	26
4.7	Failed requests during rolling updates (level A; totals over the runs of a scenario)	30
4.8	Objectives compared with the results	32
5.1	Observations of this project compared with the literature	35
5.2	Future enhancements and their reasons	36
A.1	Files of the repository that are not printed	53

LIST OF ABBREVIATIONS

API	Application Programming Interface
CI	Continuous Integration
CPU	Central Processing Unit
CSS	Cascading Style Sheets
HPA	Horizontal Pod Autoscaler
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
kind	Kubernetes in Docker
OCI	Open Container Initiative
PDB	Pod Disruption Budget
TLS	Transport Layer Security
UID	User Identifier
WSGI	Web Server Gateway Interface
YAML	YAML Ain't Markup Language

GLOSSARY OF TECHNICAL TERMS

Terms	Definition
Atomic upgrade	An upgrade started with `--atomic`: if the release does not become ready within the timeout, Helm rolls it back.
Chart	A Helm package: templates, default values and metadata that describe an application.
Checksum annotation	An annotation of the Pod template with the hash of the ConfigMap, so that a changed configuration restarts the Pods.
ConfigMap	A Kubernetes object that holds non-confidential settings as key-value pairs.
Deployment	A Kubernetes object that keeps a desired number of identical Pods running and performs rolling updates.
Evidence level	The label A to D of this report that states where and how a result was obtained.
Namespace	A Kubernetes mechanism that separates groups of objects in one cluster; one per environment here.
Pod	The smallest deployable unit of Kubernetes; one or more containers that share a network address.
Readiness probe	A periodic check; while it fails, the Pod receives no traffic from its Service.
Release, revision	A release is one installation of a chart with one set of values; every install, upgrade and rollback creates a numbered revision.
Rolling update	Step-by-step replacement of the Pods of a Deployment, controlled by `maxSurge` and `maxUnavailable`.
Secret	A Kubernetes object for confidential data; base64-encoded unless encryption at rest is configured.
Service	A Kubernetes object that gives a set of Pods a stable address and balances requests between them.
Values file	A YAML file whose entries override the defaults of a chart; one per environment here.

CHAPTER 1
INTRODUCTION

1.1 Background and Motivation

Every application that reaches users passes through several environments: development, testing, staging and production. The code is the same in all of them; the configuration is not. Replica counts, resource limits, log levels, host names, image versions and feature flags change from one environment to the next. The Twelve-Factor App method states that such configuration must be strictly separated from code [1], and empirical studies of configuration errors show why this deserves engineering discipline of its own: a wrong value can stop a perfectly correct program [2, 3].

DevOps practice sends every change through the same automated steps to every environment [4, 5]. Containers package an application with its dependencies so that it behaves identically wherever it runs [6], and Kubernetes, whose design draws on Google's Borg [7, 8], keeps the desired number of containers running and replaces them gradually during updates [9]. What Kubernetes does not provide is a convenient way to describe one application several times with controlled differences. Helm fills this gap. A *chart* holds templates, default values and metadata [10]; values files passed with `-f` and `--set` override the defaults in a fixed order [11]; and every installation is a numbered *revision* that one command can restore [12]. Studies of public charts also report outdated components and security risks [13], which is why this project fixes the versions of its image, tools and dependencies.

1.2 The Multi-Environment Configuration Problem

The simplest way to support four environments is to keep four folders of manifests. Over time a fix applied to production is forgotten in staging, a new variable is added to development only, and nobody can say which differences are intentional. Configuration code is itself defect-prone and needs the same review and testing as application code [14, 15], and public Kubernetes manifests contain security misconfigurations [16]. Most of the content of the four folders is identical and only a few values differ. Figure 1.1 contrasts the two organisations: the project treats the manifests as one *template* and the differences as *data*.

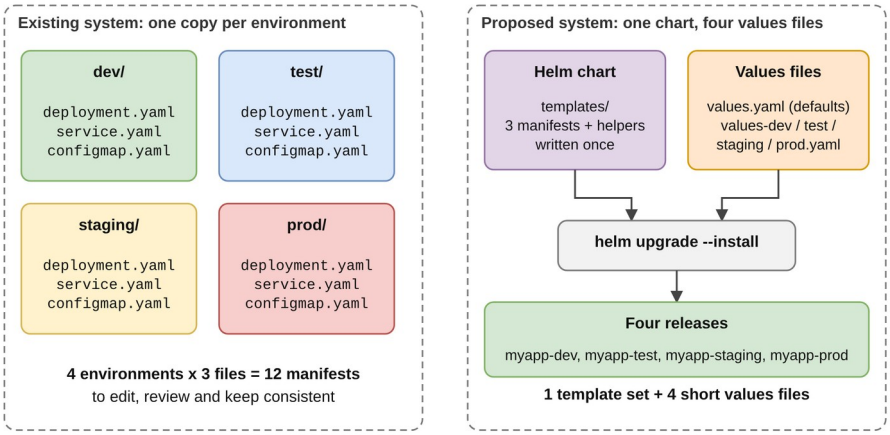


Figure 1.1 – Copied manifests per environment compared with one chart and four values files

1.3 Problem Statement

Design and implement a configuration-management solution in which one application is described once, can be deployed to development, test, staging and production with environment-specific settings, validates those settings before deployment, can be upgraded and rolled back safely, and is checked automatically on every change. The solution must be reproducible on a laptop cluster, must not contain hard-coded secrets and must not require copying manifests between environments.

1.4 Objectives of the Project

The project pursues ten objectives, taken from the project brief. Table 1.1 lists them with the artefact that realises each one and the section where it is discussed.

Table 1.1 – Project objectives and where they are addressed

No.	Objective	Realised by	Discussed in
O1	Simple web application with Flask, HTML, CSS and Bootstrap	`app/`	Section 3.4
O2	Dashboard with application name, environment, version and configuration	`index.html`, `/api/config`	Section 3.4.2, Section 4.3
O3	Four environments: development, testing, staging, production	`values-<env>.yaml`	Section 3.7.2, Section 4.2
O4	Reusable Helm chart with the required files	`helm/multi-env-app/`	Section 3.6, Section 3.8
O5	Docker configuration to containerise the application	`Dockerfile`	Section 3.5
O6	Kubernetes deployment on Minikube	`minikube-setup.sh`, `deploy.sh`	Section 3.9, Section 4.1
O7	Helm commands for install, validation, upgrade and rollback	`helm upgrade --install`, `helm rollback`	Section 3.10, Section 4.7
O8	GitHub Actions integration for automatic validation	`helm-validation.yml`	Section 3.12
O9	Testing scripts for environment-specific configuration	`tests/`, `test-environments.sh`	Section 3.11, Section 4.4
O10	No hard-coded secrets; consistent configuration	`values.schema.json`, Secret	Section 3.13, Section 4.6

1.5 Scope, Limitations and Contribution

The deployed workload is a small stateless Flask application whose behaviour depends entirely on configuration, which makes the effect of every value visible on its dashboard. The four environments are separated by namespaces in one cluster. The chart contains no Secret: the production `SECRET_KEY` is created by the deployment script and never stored in Git. Ingress, autoscaling and a GitOps controller are not used and are listed as extensions in Chapter V. The chart, scripts and tests were executed on the development machine and in GitHub Actions, where a temporary *kind* cluster [17] runs all four environments. Minikube [18] is the documented local platform and the scripts support it, but it was not executed; Section 4.1 states for every result where and how it was obtained.

The main contribution is a small, reproducible example of configuration management with Helm that a student team can read in one sitting and run on a laptop: a reusable chart, a schema-validated values interface, release management with rollback and a workflow that deploys every environment to a real cluster. The report goes beyond a demonstration with two deliberate failures (a missing image and a missing Secret) and a measurement of the requests that fail while a release is updated.

1.6 Organisation of the Report

Chapter II surveys the literature. Chapter III presents the methodology: requirements, architecture, application, container, chart, values, release management, scripts, tests and the CI workflow. Chapter IV states the evidence levels and presents the results of the cluster runs, tests and experiments. Chapter V concludes and lists future enhancements. The appendices print the core source files.

CHAPTER 2
LITERATURE SURVEY

2.1 Research Papers and Documentation

The works in Table 2.1 were reviewed for their relevance to configuration errors, configuration code, container platforms and automated delivery. The table states only what each work reports and what the project takes from it.

Table 2.1 – Surveyed literature and its consequence for the design

Source	What it reports	Consequence for this project
Wiggins, *The Twelve-Factor App* [1]	Configuration that varies between deploys must be stored in the environment, strictly separated from code	All settings are environment variables; Helm supplies them from a ConfigMap and, in production, a Secret
Yin et al., SOSP 2011 [2]	546 real misconfigurations; most are mistakes in setting parameters, a large share of them illegal values	Typed and bounded values checked by `values.schema.json`
Xu and Zhou, 2015 [3]	Survey of techniques to prevent, detect and diagnose configuration errors	Prevention (schema) and detection (lint, rendering, tests, CI) before deployment
Rahman and Williams, 2018; Rahman et al., 2019 [14, 15]	Configuration scripts are defect-prone; a mapping study of 31 infrastructure-as-code publications finds testing little studied	Chart and values are reviewed, versioned and tested like code
Rahman et al., 2023 [16]	2,039 Kubernetes manifests from 92 repositories show 11 categories of security misconfiguration, such as missing limits	Resource limits in every environment, non-root user, all capabilities dropped
Zerouali et al., 2023 [13]	9,482 Helm charts: most are unofficial and many images carry known vulnerabilities	Pinned versions, slim base image, few dependencies
Verma et al., 2015; Burns et al., 2016 [7, 8]	Borg and the lessons that led to Kubernetes, including declarative specifications	Desired state is declared; restarts, rolling updates and probes are left to the platform
Limoncelli, 2018 [19]	GitOps: desired state in version control, changed by reviewed commits	All files, including every environment's values, are in Git and checked by one workflow
Humble and Farley, 2010; Forsgren et al., 2018 [4, 5]	The deployment pipeline; automation and version control are associated with better delivery performance	GitHub Actions: tests, lint, render, build, deploy to kind
Merkel, 2014 [6]	Containers give a consistent environment from development to deployment	One image is built once and promoted unchanged

The research literature describes problems and principles; the behaviour of the tools is described in their documentation, the second class of sources. The Helm documentation defines the layering of values [11], chart tests [20], the linter [21] and the upgrade and rollback commands [12, 22]; the Kubernetes documentation describes ConfigMaps [23], Secrets [24], probes [25] and Services [26]; and the GitHub documentation defines the workflow syntax [27].

2.2 Existing Approaches to Configuration Management

Four approaches are common: copying manifests per environment; injecting settings through environment variables or files baked into images [1]; layering overlays on a shared

base, for example with Kustomize [28]; and packaging the application as a chart with layered values [11]. A GitOps controller can reconcile any of them from Git [19]. Table 2.2 compares them on the criteria that matter here.

Table 2.2 – Comparison of configuration-management approaches

Approach	Duplication	Input validation	History and rollback	Remark
Copied manifests per environment	High: every object is repeated	None beyond the API server checks	Only through Git history	Drift is likely
Environment variables or baked-in files	Low for settings, none for structure	Application-specific	Image tags only	Does not describe Kubernetes objects
Overlays and patches on a base	Low	The rendered output can be validated	Not built in; relies on Git	No release concept
Helm chart with values layering	Low: one template set	Schema in `values.schema.json`; lint	Numbered revisions; `helm rollback` [12]	Chosen approach
GitOps controller over any of these	Depends on the source	Depends on the source	Git revert and controller history	Planned extension

2.3 Research Gaps and Objective

The survey leads to four requirements: configuration must be separated from structure; every value must be validated against an explicit contract before deployment; every deployment must be recorded so that it can be reversed; and every change must pass the same automated checks (Section 3.2). The studies cited analyse existing repositories and describe the problems found in them; they do not supply a small, executable reference with the following properties, which motivate this project:

1. **A small, complete, reproducible reference.** A reusable chart, validated values, per-environment configuration, rollback and CI are rarely available together in an example that can be read in one sitting and run on a laptop.
2. **Results with a stated origin.** Reports on deployment tooling seldom say whether a result was measured on a cluster or only derived; this report labels every result with its evidence level (Section 4.1).
3. **Attention to what happens during a rollout.** A rolling update is often called “zero downtime” without measurement; the project measures the failed requests (Section 4.9).
4. **Silent failure modes.** The behaviour of a release when a prerequisite such as an image or a Secret is missing is rarely shown; the project provokes both failures (Section 4.8).

CHAPTER 3
PROPOSED METHODOLOGY

3.1 Existing and Proposed System

In the existing arrangement each environment has its own directory of manifests: a Deployment, a Service and a ConfigMap are written for development and copied, with edited values, for test, staging and production. This needs no special tooling, but differences between the copies are not visible as differences, a change must be repeated in every copy, nothing checks the edited values before the cluster accepts or rejects them, and there is no recorded history to restore. The proposed system replaces the copies with one Helm chart and four small values files; Table 3.1 compares the two. The application reads all settings from environment variables, so the same image runs in every environment, and a GitHub Actions workflow repeats the validation on every change and deploys all four environments to a temporary cluster.

Table 3.1 – Existing and proposed systems compared

Aspect	Existing system	Proposed system
Structure	One set of manifests per environment	One set of templates for all environments
Environment differences	Hidden inside the copies	Explicit in `values-<env>.yaml` (28 to 43 lines each)
Application image	Often rebuilt or edited per environment	One image `multi-env-app:1.0.0` promoted unchanged
Validation	By the API server at deploy time	Schema, lint, render and manifest checks before deploy
Deploy command	Apply several files by hand	`helm upgrade --install` with one values file
Secrets	Often written into the manifests	Never in Git; Secret created at deploy time
History and rollback	Search Git for an older file	`helm history` and `helm rollback`

3.2 System Requirements

The functional requirements follow from the objectives and the survey: (FR1) render one application for four named environments from one chart, taking every environment-specific setting from a values file; (FR2) reject unknown environments, wrong types, out-of-range values and secrets in the configuration before deployment; (FR3) show the name, environment, version and configuration on a dashboard and as JSON; (FR4) install, upgrade, test and list the revisions of a release with one command each; (FR5) restore an earlier revision with one command, and automatically after a failed upgrade; (FR6) run lint, rendering, manifest validation and a deployment test on every change; (FR7) keep secrets out of the repository, so that production reads its `SECRET_KEY` from a Kubernetes Secret. The

non-functional requirements are reproducibility on a laptop cluster, readable files, secure defaults, pinned tool versions and honest reporting of results. Table 3.2 lists the software and hardware used.

Table 3.2 – Software and hardware environment

Component	Version or specification	Purpose
Helm	v3.16.2 (pinned in CI and used locally)	Packaging, rendering, release management
Kubernetes	v1.35.0 on kind v0.31.0 (CI); manifests schema-checked for 1.31.0	Target platform; local cluster
Minikube	Any current version (documented, not executed); 2 CPUs and 4 GB, a practical minimum that was not measured	Local cluster on a laptop
Docker, Python	Docker 28.0.4 (CI), 29.9 (development machine); Python 3.12 (image, CI), 3.13 (development machine)	Image build; Flask application and tests
CI runner	GitHub-hosted ubuntu-latest (details in Section 4.1)	Runs all workflows

3.3 Block Diagram and Explanation

Figure 3.1 shows the end-to-end flow; the numbers of the explanation below are those of the figure.

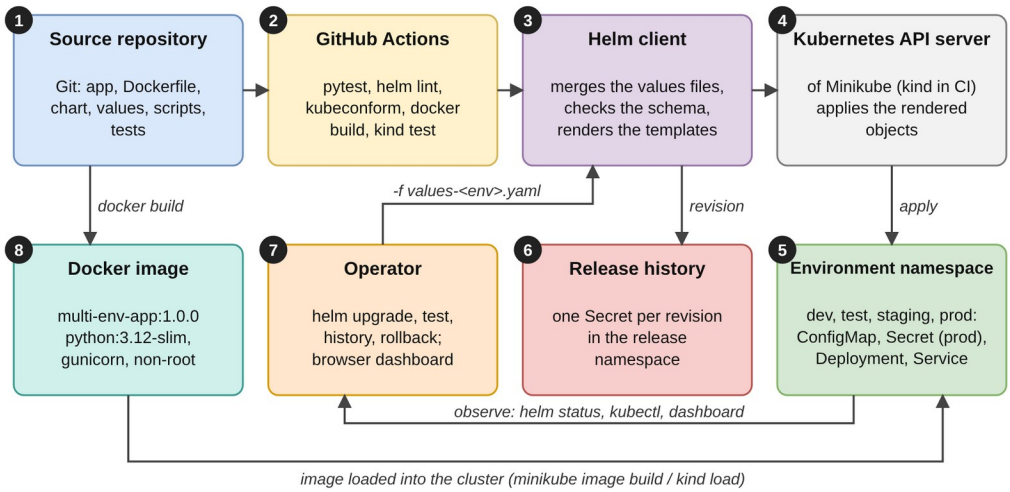


Figure 3.1 – Block diagram of the multi-environment Helm solution

1. **Source repository and GitHub Actions.** Git holds the application, `Dockerfile`, chart, scripts, tests and workflows. On every push the workflow runs the tests, lints and renders the chart for the four environments, validates the manifests, builds the image and deploys all four environments to a temporary kind cluster (Section 3.12).
2. **Helm client.** For one release the client merges `values.yaml`, one environment file and the command-line overrides, checks the result against `values.schema.json` and renders the templates into plain manifests.
3. **Kubernetes API server and namespaces.** The API server of Minikube (or kind) receives the rendered ConfigMap, Deployment and Service and the controllers create

the Pods. Each environment has its own namespace (`dev`, `test`, `staging`, `prod`); production also holds the Secret `multi-env-app-secrets`.

4. **Release history.** Helm stores every revision as a Secret of type `helm.sh/release.v1` in the namespace of the release; `helm history` and `helm rollback` read it.
5. **Operator and image.** The operator uses `helm upgrade`, `helm test`, `helm history`, `helm rollback`, `kubectl` and the dashboard. The image is built once and loaded into the cluster (`minikube image build` or `kind load`); no registry is needed.

3.4 Application Design

The application is a small Flask [29] service whose only job is to make the configuration that Helm injected visible (Figure 3.2). Its complete code is in Appendix B.

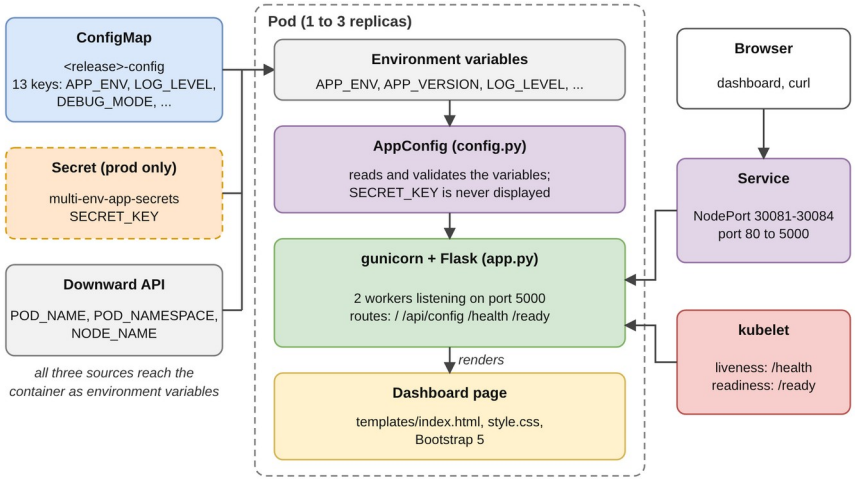


Figure 3.2 – Application inside a Pod: sources of configuration, request flow and probes

3.4.1 Routes, Configuration and Secret Handling

Four read-only routes (Table 3.3) serve people and scripts and Kubernetes. `create_app()` builds the configuration once and every route uses it, so the dashboard, the JSON and the probes cannot disagree about the environment. `AppConfig` takes a snapshot of the environment variables at start, validates `APP_ENV` against the four environments, converts booleans and integers with safe defaults and separates the settings that may be shown from the secret key. If no `SECRET_KEY` is supplied a random key is generated per process, so no key is ever committed; the dashboard and the API show only the *source* of the key (`kubernetes-secret` or `auto-generated`). Settings such as `MAX_CONNECTIONS` or `DATABASE_HOST` are displayed but not used by a database; they stand for settings that differ between environments. Table 3.4 lists the variables.

Table 3.3 – Application routes

Route	Purpose	Response	Used by
`/`	Dashboard `index.html`	HTML, 200	Browser
`/api/config`	Name, environment, version, settings (secret masked), runtime	JSON, 200	curl, `helm test`, live checks
`/health`	Liveness: the process answers	JSON, 200: `status` ok	kubelet, Docker HEALTHCHECK
`/ready`	Readiness: ready for traffic	JSON, 200: `status` ready, `environment`	kubelet, availability probe

Table 3.4 – Environment variables read by the application

Variable	Default in code	Set by
`APP_NAME`, `APP_VERSION`, `REPLICA_COUNT`	Multi-Env Config Dashboard, 1.0.0, 1	ConfigMap, from `app.name`, `app.version` (or the image tag), `replicaCount`
`APP_ENV`	development; an unknown value falls back to it with a warning	ConfigMap, from `environment`
`LOG_LEVEL`, `DEBUG_MODE`, `FEATURE_BETA_BANNER`	INFO, false, false	ConfigMap, `config.*`
`MAX_CONNECTIONS`, `CACHE_TTL_SECONDS`, `DATABASE_HOST`, `API_BASE_URL`	10, 60, localhost, http://localhost:5000/api	ConfigMap, `config.*` (illustrative settings)
`HELM_RELEASE`, `HELM_CHART_VERSION`	not-deployed-with-helm, n/a	ConfigMap, from the release and the chart
`SECRET_KEY`	random per process; shown only as a masked value	Secret `multi-env-app-secrets` (production)
`POD_NAME`, `POD_NAMESPACE`, `NODE_NAME`, `PORT`	local, local, local, 5000	Downward API [30]; Deployment

3.4.2 Dashboard and Probes

The dashboard is a Jinja2 template styled with Bootstrap 5 [31] from a CDN and a short `style.css`; it is rendered on the server and needs no JavaScript. Figure 3.3 labels its parts: (1) the navigation bar with the application name and the badge of the environment; (2) cards for the name, environment, version and desired replicas; (3) the configuration table, which is the content of the ConfigMap as the application sees it; (4) the Kubernetes runtime panel with Pod, namespace, node, release and chart version; (5) links to `/api/config`, `/health` and `/ready`; and (6) the footer. The colour of the bar, cards and table header comes from the environment (green for development, cyan for testing, yellow for staging, striped red for production), so that the audience can tell at a glance which environment is shown. Kubernetes calls `/health` for the liveness probe and `/ready` for the readiness probe [25]; the logger level is taken from `LOG_LEVEL`, so the effect of the setting is visible in the logs (Section 4.3).

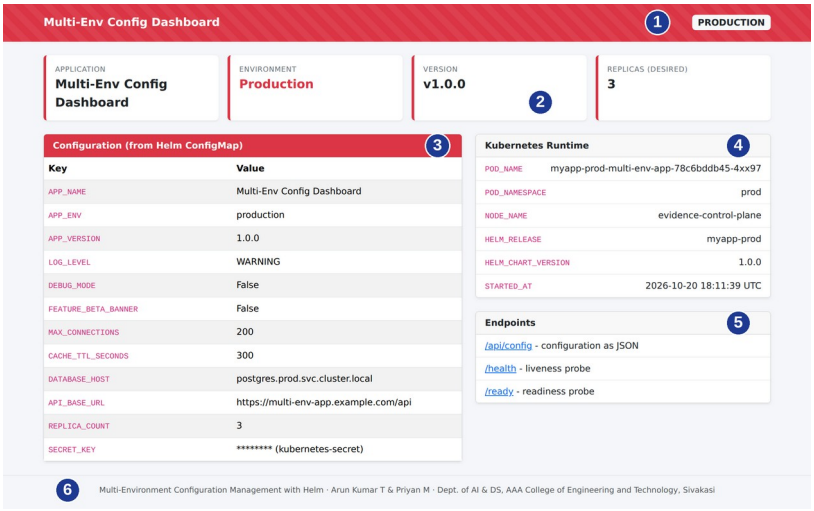


Figure 3.3 – Parts of the dashboard (production release)

3.5 Containerisation

The `Dockerfile` builds one image that is used in all four environments (Figure 3.4). Dependencies are installed before the code is copied, so that Docker can reuse the cached dependency layer when only the code changes [32]. The container runs as the unprivileged user 10001, which matches the `securityContext` of the chart; a HEALTHCHECK lets Docker report its state outside Kubernetes; and gunicorn [33] with two workers replaces the Flask development server, started with `exec` so that it receives the termination signal of Kubernetes. The file is printed in Appendix C.

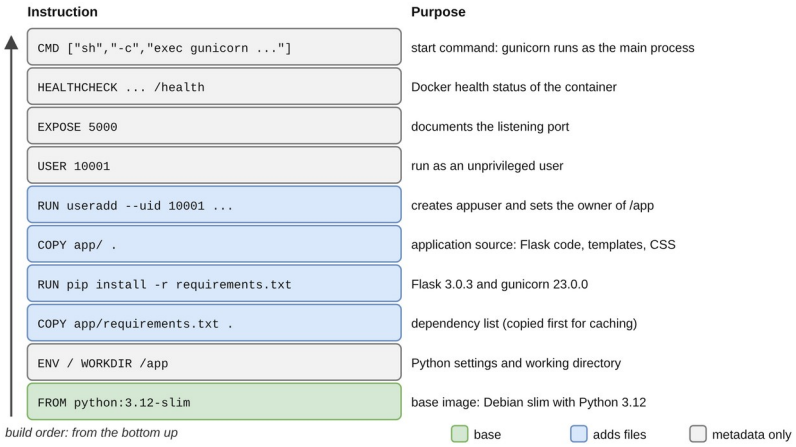


Figure 3.4 – Layered structure of the application image (instruction and purpose)

3.6 Helm Chart Design

The chart `multi-env-app` (`apiVersion: v2`, chart version and application version 1.0.0 [10]) consists of `Chart.yaml`, `values.yaml`, four environment files, `values.schema.json`, `_helpers.tpl`, templates for the ConfigMap, Deployment and Service, a

test Pod and `NOTES.txt`; the directory tree is in Appendix A. Names and labels are produced once in `_helpers.tpl` and included by every template. An object is named after the release (`myapp-dev-multi-env-app`), and two label sets exist: the *selector labels*, which must not change between upgrades because the selector of a Deployment is immutable, and the *common labels*, which add chart, version, manager and environment.

3.7 Values and Environment-Specific Configuration

3.7.1 Layering of Values

Helm builds the values in layers [11]: `values.yaml` has the lowest precedence, files passed with `-f` override it in the order given, and `--set` has the highest precedence; maps are merged key by key, so an environment file mentions only what differs (Figure 3.5). `deploy.sh` passes exactly one environment file, so `-f values-prod.yaml` is all that distinguishes a production release from a development release. Table 3.5 resolves six values for a staging release started with `--set replicaCount=3 --set config.LOG_LEVEL=DEBUG`; the last column was produced with `helm template`. Settings under `config` are strings because they end up in a ConfigMap, so numbers must be given with `--set-string`, as `--set config.CACHE_TTL_SECONDS=130` is rejected by the schema; Helm itself recommends clearly typed values [34].

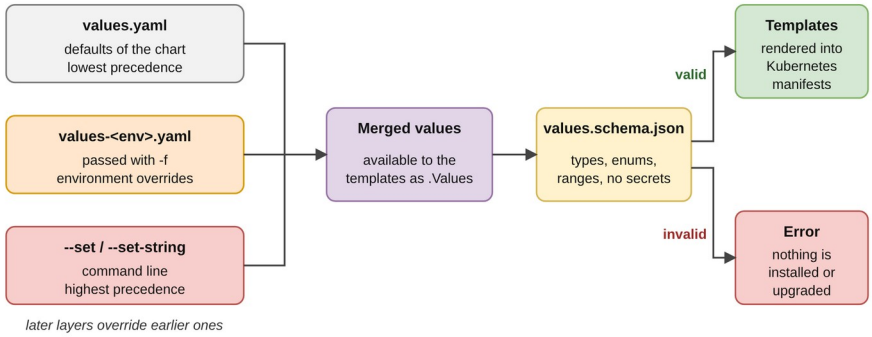


Figure 3.5 – Values layering and schema validation

Table 3.5 – Effective values of a staging release with two command-line overrides

Value	values.yaml	values-staging.yaml	Command line	Effective value
`replicaCount`	1	2	3	3 (command line)
`config.LOG_LEVEL`	INFO	INFO	DEBUG	DEBUG (command line)
`config.MAX_CONNECTIONS`	10	50	none	50 (file)
`config.FEATURE_BETA_BANNER`	false	false	none	false
`service.nodePort`	empty	30083	none	30083 (file)
`image.tag`	empty	not set	none	empty, so `1.0.0` from `appVersion`

3.7.2 Configuration of the Four Environments

Table 3.6 lists the settings that differ between the environments, read from the values files. Development is the smallest and most verbose; testing adds information-level logs and the beta banner; staging has the flags of production, two replicas and larger limits; production has three replicas, warning-level logs, the largest limits and its secret key from a Kubernetes Secret. The values are design choices for demonstration; a real project would tune them from measurements.

Table 3.6 – Environment-specific configuration (values that differ)

Setting	dev	test	staging	prod
Release / namespace	myapp-dev / dev	myapp-test / test	myapp-staging / staging	myapp-prod / prod
`environment`	development	testing	staging	production
Replicas	1	1	2	3
`LOG_LEVEL`	DEBUG	INFO	INFO	WARNING
`DEBUG_MODE`	true	false	false	false
`FEATURE_BETA_BANNER`	true	true	false	false
`MAX_CONNECTIONS`	5	10	50	200
`CACHE_TTL_SECONDS`	10	30	120	300
CPU request / limit	25m / 100m	50m / 150m	100m / 250m	100m / 500m
Memory request / limit	64Mi / 128Mi	64Mi / 128Mi	96Mi / 192Mi	128Mi / 256Mi
NodePort	30081	30082	30083	30084
Secret	none (key generated)	none (key generated)	none (key generated)	multi-env-app-secrets
Lines in the values file	28	28	28	43

3.7.3 Schema Validation of Values

`values.schema.json` is a JSON Schema (draft-07) [35] that Helm checks against the merged values on install, upgrade, lint and template [10]. Table 3.7 summarises its rules. It requires the top-level keys `environment`, `replicaCount`, `image`, `app`, `config` and `service`, and it forbids the keys `SECRET_KEY`, `PASSWORD`, `DB_PASSWORD` and `API_TOKEN` inside `config`, so that a secret cannot end up in a ConfigMap by mistake.

Table 3.7 – Rules of values.schema.json

Value	Rule	Purpose
`environment`	One of development, testing, staging, production	An unknown environment cannot be deployed
`replicaCount`	Integer, 1 to 10	No empty or runaway Deployments
`config.LOG_LEVEL`	DEBUG, INFO, WARNING, ERROR or CRITICAL	Levels the application understands
`config.DEBUG_MODE`, `config.FEATURE_BETA_BANNER`	The string `true` or `false`	ConfigMap values are strings
`config.MAX_CONNECTIONS`, `config.CACHE_TTL_SECONDS`	String of digits	Numbers the application can parse
Keys of `config`	Upper-case letters, digits and underscores	Valid environment-variable names

Value	Rule	Purpose
SECRET_KEY, PASSWORD, DB_PASSWORD, API_TOKEN in config	Forbidden	No secrets in a ConfigMap or in Git
image.pullPolicy, service.type	Always, IfNotPresent, Never; ClusterIP, NodePort, LoadBalancer	Valid Kubernetes values
containerPort, service.port, service.nodePort	Integer 1 to 65535; NodePort empty or 30000 to 32767	Valid ports

3.8 Helm Templates

Three templates create the ConfigMap, Deployment and Service, a fourth creates the Pod run by `helm test`, and `NOTES.txt` prints a message after an install. All read their settings from `.Values`, so no environment name or number appears in a template.

ConfigMap. Six keys are fixed (`APP_NAME`, `APP_ENV`, `APP_VERSION`, `REPLICA_COUNT`, `HELM_RELEASE`, `HELM_CHART_VERSION`); the others are generated with a `range` loop over the `config` map, each value converted to a quoted string. A new setting needs no template change: it is added to `config` and appears as an environment variable. The production ConfigMap has 13 keys (Section 4.2).

Deployment. The container receives the ConfigMap with `envFrom`; if `existingSecret` is set, a `secretRef` is added to the same list. `POD_NAME`, `POD_NAMESPACE` and `NODE_NAME` come from the Downward API. Probes, resources, security contexts and the strategy are copied from the values with `toYaml`, so an environment can change them without a template change. Variables from a ConfigMap are read only when a container starts, so the template stores the SHA-256 checksum of the rendered ConfigMap in the annotation `checksum/config` of the Pod template; when a value in `config` changes, the Pod template changes and the Deployment performs a rolling update. The pytest case `test_config_change_changes_checksum` verifies this.

Code Snippet – Checksum annotation (templates/deployment.yaml, lines 21–23)

```
    annotations:
      # Roll the pods automatically whenever the ConfigMap content changes.
      checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . |
sha256sum }}
```

Service and test. The Service selects only the Pods of its own release through the selector labels; its target port is the *name* `http`, so a change of `containerPort` needs no change in the Service. A fixed `nodePort` (30081 to 30084) is rendered only for the type NodePort [26]. The test Pod has the annotation `helm.sh/hook: test`, is created only by `helm test` [20], requests `/health` and `/api/config` with `busybox` and checks that the

answer contains the environment name of the release, so a production release fails the test if it answers with another environment.

3.8.1 Rolling-Update Strategy

The strategy is a value: `values.yaml` sets `RollingUpdate` with `maxSurge: 1` and `maxUnavailable: 0`. For a Deployment of N replicas the controller keeps the number of available Pods A(t) and the total number of Pods P(t) within the bounds Equation (3.1) and Equation (3.2) [9], where *s* is `maxSurge` and *u* is `maxUnavailable`.

$$N - u \leq A(t) \tag{3.1}$$

$$P(t) \leq N + s \tag{3.2}$$

With $u = 0$ and $s = 1$ an old Pod is removed only after a new one is ready, so A(t) never falls below N and P(t) never exceeds N + 1; in production this means at least three and at most four Pods. The alternative $s = 0$ and $u = 1$ allows A(t) to fall to N − 1, which is zero for a single replica. The bounds describe Pods, not requests; Section 4.9 measures what they mean for a client. Table 3.8 gives the size of the rendered manifests (four objects per environment: ConfigMap, Service, Deployment and test Pod), used in Section 4.5.

Table 3.8 – Rendered manifests and size of the environment files

Environment	Rendered lines	Values that differ from `values.yaml`	Lines in the values file
dev	177	11	28
test	177	8	28
staging	177	12	28
prod	179	13	43

16 settings are changed by at least one environment; 139 of the 177 lines of the development manifest occur in each of the other three manifests. Production is two lines longer because of the `secretRef`.

3.9 Kubernetes Topology and Objects

Figure 3.6 shows the cluster as deployed by `scripts/deploy.sh all`: one cluster, four namespaces, and in each one release with its ConfigMap, Deployment, Service and Pods. The objects of the production release are the namespace `prod`; the ConfigMap `myapp-prod-multi-env-app-config`; the Secret `multi-env-app-secrets`, which `deploy.sh` creates and the chart only refers to; the Deployment `myapp-prod-multi-env-app` with its ReplicaSet (one for every distinct Pod template) and three Pods; the Service with the NodePort 30084; one Secret of type `helm.sh/release.v1` for every revision, such as `sh.helm.release.v1.myapp-prod.v1` [10, 36]; and the test Pod that `helm test` creates. All objects of a release carry the labels `app.kubernetes.io/instance` and `app.kubernetes.io/name`; the live objects are shown in Section 4.2.

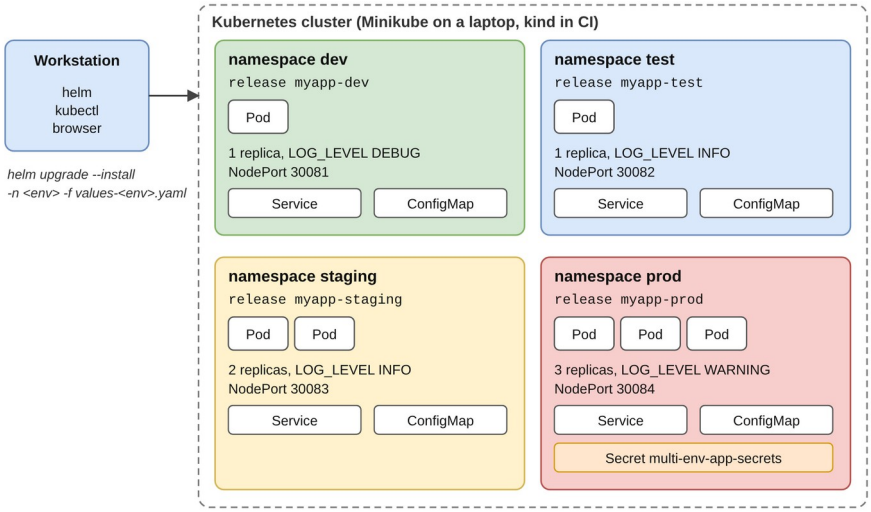


Figure 3.6 – Cluster topology: one namespace and one Helm release per environment

3.10 Release Management with Helm

A *release* is one installation of a chart with one set of values; the releases are named `myapp-<env>` and installed into the namespace `<env>` (Figure 3.7). `helm upgrade --install` creates the release if it does not exist and upgrades it otherwise; every change creates a numbered revision, and `helm rollback` creates another revision with the content of an earlier one [12]. `--wait` makes Helm return only after the Pods are ready, and `--atomic` implies `--wait` and rolls the release back if the upgrade does not become ready within the timeout [22]. The upgrade script does not use `--reuse-values`: the values file of the environment is always the source of truth. Table 3.9 lists the commands used.

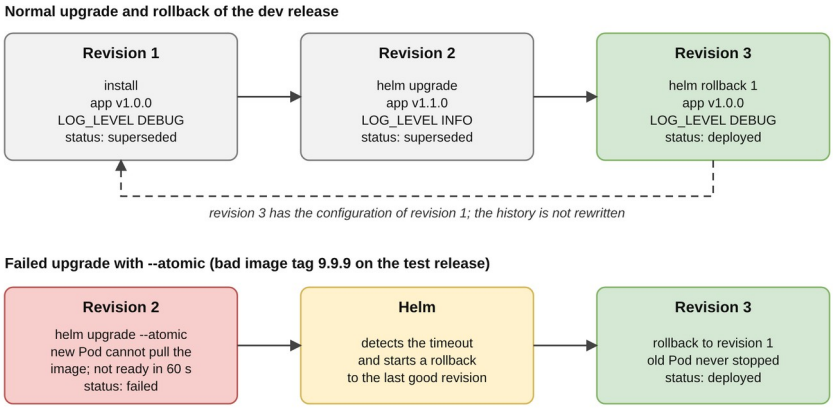


Figure 3.7 – Lifecycle of a Helm release

Table 3.9 – Helm commands used in the project

Command	Purpose	Used by
`helm lint ./helm/multi-env-app --strict -f values-dev.yaml`	Static checks; enforces the schema; warnings are errors	`validate-helm.sh`, CI, pytest
`helm template myapp-dev ./helm/multi-env-app -f values-dev.yaml`	Renders the manifests without a cluster	`validate-helm.sh`, pytest
`helm upgrade --install myapp-dev ./helm/multi-env-app -n dev -f values-dev.yaml --atomic --wait --timeout 5m`	Installs or upgrades and waits until ready	`deploy.sh`
`helm test myapp-dev -n dev`	Runs the test Pod of the release	`test-environments.sh`
`helm history`, `helm get values`, `helm list -A`	Revisions, supplied or computed values, releases	Demo script, evidence capture
`helm rollback myapp-dev <previous> -n dev --wait --timeout 5m`	New revision with the content of the previous revision (revision 1 in the demonstration)	`upgrade-rollback-demo.sh`
`helm uninstall myapp-dev -n dev`	Removes a release and its objects	`cleanup.sh`

3.11 Automation Scripts and Testing Strategy

Nine shell scripts in `scripts/` share the settings of `common.sh` (namespace, values file, release name, expected `APP_ENV` and NodePort of each environment), stop at the first error and check that the tools they need are installed (Table 3.10). A session on a laptop runs `minikube-setup.sh`, `validate-helm.sh`, `deploy.sh all`, `test-environments.sh all`, `upgrade-rollback-demo.sh dev` and `cleanup.sh` in this order. `validate-helm.sh` needs no cluster.

Table 3.10 – Scripts of the project

Script	Lines	Purpose
`common.sh`	43	Shared settings and helper functions
`minikube-setup.sh`	25	Starts Minikube (2 CPUs, 4096 MB), builds the image inside it, creates the namespaces
`deploy.sh <env|all>`	49	Creates the namespace and the production Secret; runs `helm upgrade --install`
`validate-helm.sh`	45	Lint, render and kubeconform check of four environments, no cluster
`test-environments.sh`	68	26 live checks of the deployed releases, including `helm test`
`upgrade-rollback-demo.sh`	50	Upgrade to 1.1.0, history, rollback, verify
`docker-demo.sh`, `cleanup.sh`	63, 12	Four environments as plain Docker containers; uninstall and delete the namespaces
`capture-evidence.sh`	238	Records the evidence of this report on a cluster (Section 4.1)

The project is tested at four levels (Table 3.11). The first three run without a cluster and give feedback in seconds; the fourth needs a cluster. All 62 pytest cases [37] pass on the development machine and in CI, and the 26 live checks pass on kind. The cases that need Helm are skipped, not failed, when the binary is absent. `test_production_hardening` states the rules for production as assertions, so an edit of `values-prod.yaml` that enables debug mode or removes the Secret fails before it reaches a cluster.

Table 3.11 – Test levels

Level	Tool and file	Cases	Examples
1. Application	pytest, `test_app.py`	15	Dashboard shows name, environment and version; probes answer; the secret is masked; an unknown environment falls back to development
2. Static configuration	pytest, `test_helm_config.py`	32	Required files; same `config` keys in every environment; unique NodePorts in range; production hardening; no secret-like key has a value
3. Chart behaviour	pytest, `test_helm_config.py`	15	`helm lint --strict` for four environments; rendered objects match the values; checksum changes with the config; five invalid values rejected
4. Live cluster	`test-environments.sh`, `helm test`	26	Release deployed; replicas ready; `APP_ENV`, `REPLICA_COUNT`; `/health`; production debug off and secret from a Secret; four `helm test` runs

3.12 Continuous Integration with GitHub Actions

The workflow `helm-validation.yml` runs on every push and pull request and on demand. It has four jobs (Table 3.12, timeline in Section 4.10); the first three run in parallel and the fourth, which deploys to a cluster, starts only if all three pass (`needs:`). The versions of Helm (v3.16.2) and kubeconform (v0.6.7) [38] and the Kubernetes version of the manifest schemas (1.31.0) are fixed in the `env:` block, so that a new tool release cannot change the result for an unchanged commit [27]. A second workflow, `capture-evidence.yml`, records the evidence of this report on a kind cluster and commits it with `[skip ci]` (Section 4.1).

Table 3.12 – Jobs of the workflow helm-validation.yml

Job	Steps	Failure means
Unit and configuration tests	Install dependencies; `python -m pytest -v` (62 cases)	Application, values or chart logic is wrong
Helm lint, template, kubeconform	`validate-helm.sh` for four environments; upload the rendered manifests	Schema violation, template error or invalid Kubernetes object
Docker build and smoke test	Build the image; run it with `APP_ENV=testing`; check `/health`, `/api/config` and the dashboard	The image does not build or start
Deploy to a kind cluster	Create cluster; load image; `deploy.sh all`; `test-environments.sh all`; `upgrade-rollback-demo.sh dev`	A release is not ready, a live check fails, or an upgrade or rollback fails

3.13 Security and Secret Handling

Table 3.13 lists the risks that the design addresses, the measure for each and where it can be checked. The project is a teaching example and does not claim to be a hardened production system.

Table 3.13 – Security measures

Risk	Measure	Where
Secret committed to Git	No secret in any values file; production reads `SECRET_KEY` from the Secret that `deploy.sh` creates with a random key	`values-prod.yaml`, `deploy.sh`, `test_no_hardcoded_secrets`
Secret placed in a ConfigMap	The schema forbids `SECRET_KEY`, `PASSWORD`, `DB_PASSWORD` and `API_TOKEN` in `config`	`values.schema.json`
Secret shown by the dashboard or API	The key is replaced by `******** (<source>)`	`config.py`, `test_secret_key_is_never_exposed`
Container runs as root	UID 10001, `runAsNonRoot`, no privilege escalation, all capabilities dropped	`Dockerfile`, `values.yaml`
Resource exhaustion	CPU and memory requests and limits in every environment	`values-*.yaml`
A bad release stays in service	`--atomic` rollback; the readiness probe keeps new Pods out of service until they answer	`deploy.sh`, `values.yaml`
Moving dependencies	Pinned Flask 3.0.3, gunicorn 23.0.0, Helm, kubeconform and schema versions; slim base image	`requirements.txt`, workflow `env:`

Kubernetes Secrets are only base64-encoded by default and can be read by anyone who may read Secrets in the namespace, so encryption at rest, role-based access control and an external secret manager are needed in a real cluster [24]. The image was not scanned for vulnerabilities and the base image is selected by tag, not digest (Chapter V).

3.14 Resource Requirements of the Four Environments

Table 3.14 multiplies the requests and limits of Table 3.6 by the number of replicas (level C: computed, not measured). All four environments request 0.575 CPU and 704 MiB, and their limits add up to 2.25 CPU and 1,408 MiB. The requests fit easily into the Minikube cluster of 2 CPUs and 4096 MB that `minikube-setup.sh` creates, which must also run the Kubernetes system Pods (not measured). The limits exceed two CPUs; this is normal, because a limit is a ceiling that is not reserved, but all Pods could not run at their limits at the same time.

Table 3.14 – Resources requested by the four environments (m = millicores, Mi = mebibytes)

Environment	Replicas	CPU request / limit per Pod	Memory request / limit per Pod	Total requests	Total limits
dev	1	25 m / 100 m	64 Mi / 128 Mi	25 m, 64 Mi	100 m, 128 Mi
test	1	50 m / 150 m	64 Mi / 128 Mi	50 m, 64 Mi	150 m, 128 Mi
staging	2	100 m / 250 m	96 Mi / 192 Mi	200 m, 192 Mi	500 m, 384 Mi
prod	3	100 m / 500 m	128 Mi / 256 Mi	300 m, 384 Mi	1,500 m, 768 Mi
All four	**7**			**575 m, 704 Mi**	**2,250 m, 1,408 Mi**

CHAPTER 4
RESULTS AND DISCUSSION

4.1 Basis of the Results

Every result in this chapter carries one of four evidence levels (Table 4.1), and the text states the level where a result is introduced.

Table 4.1 – Evidence levels used in this report

Level	How the result was obtained	Used for
A Cluster run	Executed on a kind cluster in GitHub Actions; output in `docs/evidence/` (`capture-evidence.yml`, run 2) or from the validation workflow (run 6)	Deployments, dashboards, live checks, upgrade, rollback, failures, availability, CI timings
B Local run	Executed on the development machine (Python 3.13, Helm v3.16.2, kubeconform v0.6.7) without a cluster	62 pytest cases, `validate-helm.sh`, schema rejection tests
C Derived	Computed from level A or B data or from the repository files	Line counts, break-even analysis, failure rates, resource totals
D Documented only	Described in the README and supported by the scripts, but not executed	Minikube on a laptop, Windows, cloud clusters

The level A results were recorded on a GitHub-hosted runner (`ubuntu-latest`, Linux 7.0.0, AMD EPYC 7763, 4 vCPUs, 15 GiB) with Docker 28.0.4, kind v0.31.0 (one control-plane node, containerd 2.2.0), Kubernetes v1.35.0, Helm v3.16.2 and headless Google Chrome 155 for the screenshots. The dashboards are screenshots of pages served by the Pods through `kubectl port-forward`, not mock-ups, and the terminal figures are rendered from the text files in `docs/evidence/` (omissions are stated in the captions). The cluster has a single node. **Not** executed were: a run on Minikube (the scripts support it), a run on Windows, a multi-node or cloud cluster, a load test, a vulnerability scan of the image and tests with other browsers.

4.2 Deployment of the Four Environments

`scripts/deploy.sh all` created the four namespaces and installed the four releases; in the validation workflow this took 40 s, and in the evidence run every release reached the status *deployed* at revision 1 (Figure 4.1). Table 4.2 summarises the objects of the four releases as the cluster reported them.

bash - helm

```
$ helm list -A
NAME           NAMESPACE  REVISION  STATUS    CHART                APP VERSION
myapp-dev      dev        1         deployed  multi-env-app-1.0.0  1.0.0
myapp-prod     prod       1         deployed  multi-env-app-1.0.0  1.0.0
myapp-staging  staging    1         deployed  multi-env-app-1.0.0  1.0.0
myapp-test     test       1         deployed  multi-env-app-1.0.0  1.0.0
```

Figure 4.1 – Helm releases after `scripts/deploy.sh all` (`helm list -A`, update time omitted)

Table 4.2 – Releases of the four environments in the cluster (level A)

Environment	Namespace	Release	Ready / desired	Pods	NodePort	ConfigMap keys
Development	dev	myapp-dev	1/1	1	30081	13
Testing	test	myapp-test	1/1	1	30082	13
Staging	staging	myapp-staging	2/2	2	30083	13
Production	prod	myapp-prod	3/3	3	30084	13

Two Secrets exist in the production namespace (Figure 4.2). `multi-env-app-secrets` holds the `SECRET_KEY` and was created by `deploy.sh`, not by the chart; `sh.helm.release.v1.myapp-prod.v1` is the record of revision 1 that Helm stores and `helm rollback` reads.

bash - kubectl

```
$ kubectl get secret -n prod
NAME                                TYPE                 DATA   AGE
multi-env-app-secrets               Opaque               1      9s
sh.helm.release.v1.myapp-prod.v1    helm.sh/release.v1   1      9s
```

Figure 4.2 – Secrets of the production namespace (`kubectl get secret -n prod`)

Figure 4.3 shows the production Deployment as described by Kubernetes. It confirms that the values reached the cluster: three replicas; the strategy `RollingUpdate` with 0 maximum unavailable and 1 maximum surge; the checksum annotation; the image `multi-env-app:1.0.0`; the resource requests and limits of production; the two probes; the environment taken from the ConfigMap *and* from the Secret `multi-env-app-secrets`; and the Downward API variables.

bash - kubectl describe

```
$ kubectl describe deployment myapp-prod-multi-env-app -n prod
Name:                   myapp-prod-multi-env-app
Namespace:              prod
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
RollingUpdateStrategy:  0 max unavailable, 1 max surge
  Annotations:  checksum/config: a36e621c071f2c2bd77b55dd9aff5a6ff345417abb758870d2f060df7b50e7cb
    Image:      multi-env-app:1.0.0
    Port:       5000/TCP (http)
    Limits:
      cpu:     500m
      memory:  256Mi
    Requests:
      cpu:      100m
      memory:   128Mi
    Liveness:   http-get http://:http/health delay=5s timeout=1s period=10s #success=1 #failure=3
    Readiness:  http-get http://:http/ready delay=3s timeout=1s period=5s #success=1 #failure=3
    Environment Variables from:
      myapp-prod-multi-env-app-config  ConfigMap  Optional: false
      multi-env-app-secrets            Secret     Optional: false
    Environment:
      PORT:           5000
      POD_NAME:        (v1:metadata.name)
      POD_NAMESPACE:   (v1:metadata.namespace)
      NODE_NAME:       (v1:spec.nodeName)
```

Figure 4.3 – Deployment of the production environment (`kubectl describe deployment`, selected lines)

4.3 Dashboards of the Four Environments

Figure 4.4 shows the upper part of the four dashboards, and Figure 4.5 the complete production dashboard. They are produced by the same image and the same templates; only the values file differs. The colour of the banner, the cards and the table header identifies the environment (green, cyan, yellow and striped red); development and testing show the beta banner because `FEATURE_BETA_BANNER` is true there. The replica cards show 1, 1, 2 and 3.

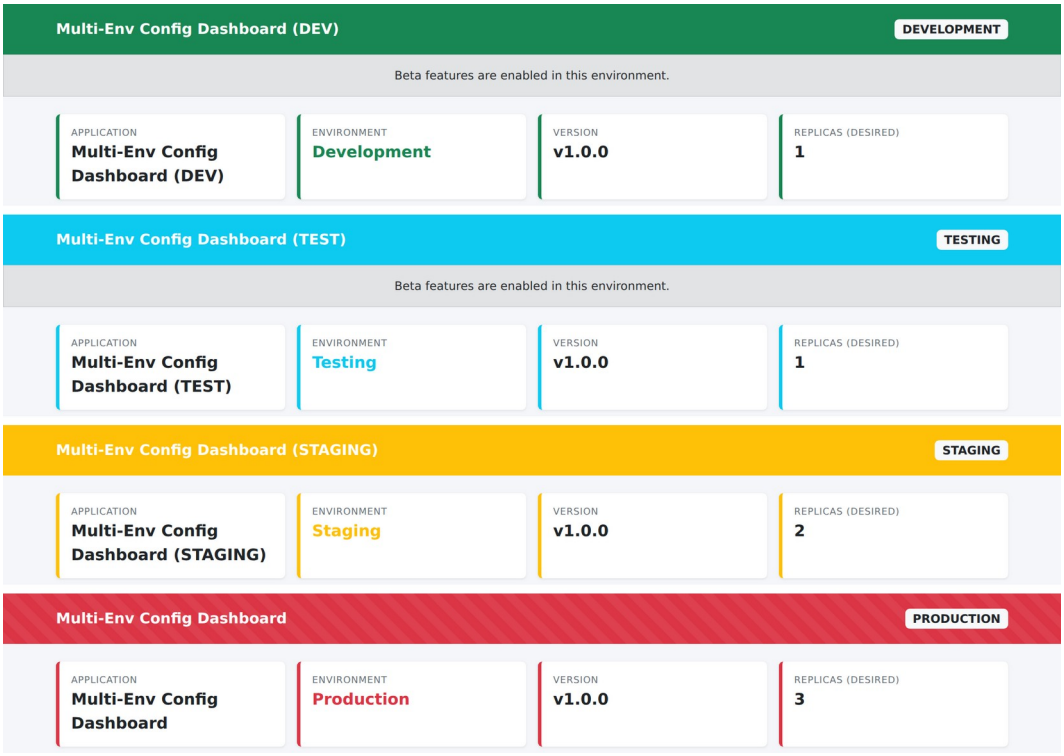


Figure 4.4 – Upper part of the dashboards of development, testing, staging and production (level A)

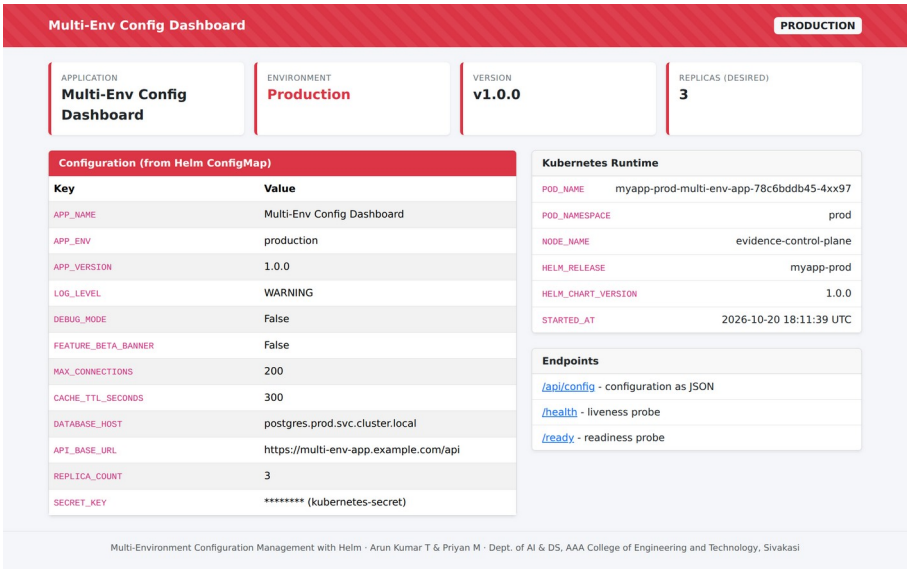


Figure 4.5 – Dashboard of the production environment, served by the Pod myapp-prod-multi-env-app-78c6bddb45-4xx97

The JSON that every Pod returns from `/api/config` was compared, value by value, with the values that Helm merges from `values.yaml` and the environment file: 11 settings in each of the four environments (44 comparisons) plus the source of the secret key, the release name and the namespace (12 comparisons). All 56 comparisons were equal (Table 4.3), which closes the chain from the file to the running container. The `SECRET_KEY` is shown only as a masked value with its source: *auto-generated* in development, testing and staging, and *kubernetes-secret* in production.

Table 4.3 – Settings reported by the running Pods through /api/config (level A)

Setting	dev	test	staging	prod
APP_NAME	Multi-Env Config Dashboard (DEV)	Multi-Env Config Dashboard (TEST)	Multi-Env Config Dashboard (STAGING)	Multi-Env Config Dashboard
APP_ENV	development	testing	staging	production
APP_VERSION	1.0.0	1.0.0	1.0.0	1.0.0
LOG_LEVEL	DEBUG	INFO	INFO	WARNING
DEBUG_MODE	true	false	false	false
FEATURE_BETA_BANNER	true	true	false	false
MAX_CONNECTIONS	5	10	50	200
CACHE_TTL_SECONDS	10	30	120	300
DATABASE_HOST	postgres.dev.svc.cluster.local	postgres.test.svc.cluster.local	postgres.staging.svc.cluster.local	postgres.prod.svc.cluster.local
API_BASE_URL	http://dev.multi-env-app.local/api	http://test.multi-env-app.local/api	https://staging.multi-env-app.example.com/api	https://multi-env-app.example.com/api
REPLICA_COUNT	1	1	2	3
SECRET_KEY	(auto-generated)	(auto-generated)	(auto-generated)	(kubernetes-secret)
HELM_RELEASE	myapp-dev	myapp-test	myapp-staging	myapp-prod
POD_NAMESPACE	dev	test	staging	prod

The effect of `LOG_LEVEL` is visible in the logs (Figure 4.6): the development Pod, with the level DEBUG, prints the start-up message of the application and the access log of gunicorn; the production Pod, with the level WARNING, prints only the messages of gunicorn and the access log, and the start-up message of the application is suppressed.

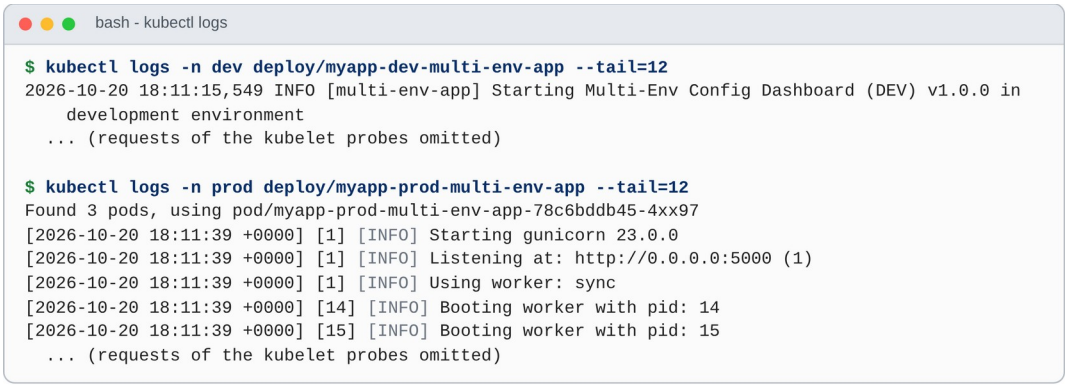


Figure 4.6 – Logs of a development Pod and a production Pod (`kubectl logs`, probe requests omitted)

4.4 Test Cases and Results

Table 4.4 lists the test cases and their results; all automated cases passed. The runs behind TC-01 to TC-06 are described below.

Table 4.4 – Test cases and results

ID	Test	Expected result	Result	Level
TC-01	Application tests (`tests/test_app.py`)	15 cases pass	15 passed	B, A
TC-02	Static configuration tests (values files)	32 cases pass	32 passed	B, A
TC-03	Chart tests with Helm (lint, render, checksum, schema)	15 cases pass	15 passed	B, A
TC-04	`validate-helm.sh`: lint, render and kubeconform for 4 environments	No error; `APP_ENV` rendered correctly; 4 of 4 objects valid per environment	Passed for all four environments	B, A
TC-05	Docker smoke test in CI	Container starts; `/health`, `/api/config` and the dashboard answer	Job succeeded	A
TC-06	Live checks `test-environments.sh all`, including `helm test`	26 checks pass; test Pod phase Succeeded	26 passed, 0 failed; 4 of 4 `helm test` succeeded	A
TC-07	Schema rejection of invalid values	14 invalid inputs are rejected before deployment	14 of 14 rejected	B
TC-08	Upgrade and rollback of development	Revision 2 shows 1.1.0 and INFO; revision 3 restores 1.0.0 and DEBUG	As expected	A
TC-09	Upgrade to a non-existent image with `--atomic`	Rollback to the previous revision; old Pod keeps running	As expected	A
TC-10	Production installed without its Secret	Pods do not start; the cause is visible in the events	CreateContainerConfigError; Secret not found	A
TC-11	Availability during rolling updates (13 runs)	Measured, not asserted	Failed requests found; see Section 4.9	A
TC-12	Validation workflow	All four jobs succeed	Succeeded (run 6)	A

Unit and configuration tests (TC-01 to TC-03). All 62 pytest cases passed on the development machine in about one and a half seconds (Figure 4.7); the CI job runs the same command with Helm installed, so the cases that need Helm are run there as well.

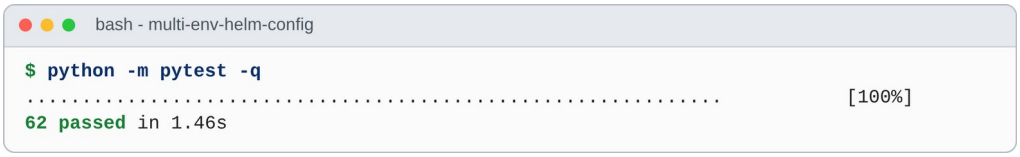


Figure 4.7 – Result of the pytest run on the development machine

Validation without a cluster (TC-04). `validate-helm.sh` lints, renders and validates each environment (Figure 4.8). All twelve stages succeeded: `helm lint --strict` reported no failed chart, every environment rendered the expected `APP_ENV`, and kubeconform found four of four objects valid in each environment.

```
bash - multi-env-helm-config
$ ./scripts/validate-helm.sh | grep -E '==>|linted|APP_ENV|Summary|validated'
==> [dev] helm lint --strict
1 chart(s) linted, 0 chart(s) failed
==> [dev] helm template
APP_ENV correctly rendered as development
==> [dev] kubeconform
Summary: 4 resources found in 1 file - Valid: 4, Invalid: 0, Errors: 0, Skipped: 0
==> [test] helm lint --strict
1 chart(s) linted, 0 chart(s) failed
==> [test] helm template
APP_ENV correctly rendered as testing
==> [test] kubeconform
Summary: 4 resources found in 1 file - Valid: 4, Invalid: 0, Errors: 0, Skipped: 0
==> [staging] helm lint --strict
1 chart(s) linted, 0 chart(s) failed
==> [staging] helm template
APP_ENV correctly rendered as staging
==> [staging] kubeconform
Summary: 4 resources found in 1 file - Valid: 4, Invalid: 0, Errors: 0, Skipped: 0
==> [prod] helm lint --strict
1 chart(s) linted, 0 chart(s) failed
==> [prod] helm template
APP_ENV correctly rendered as production
==> [prod] kubeconform
Summary: 4 resources found in 1 file - Valid: 4, Invalid: 0, Errors: 0, Skipped: 0
All environments validated successfully.
```

Figure 4.8 – Output of `scripts/validate-helm.sh` (the three stages of each environment)

Live checks (TC-06). `test-environments.sh all` ran 26 checks on the cluster (Table 4.5): six for each of development, testing and staging and eight for production, which has two more checks, one for the debug mode and one for the source of the secret key. The last check of every environment is `helm test`, whose Pod requests `/health` and `/api/config` and compares the environment name; all four passed. The complete output is in `docs/evidence/05-live-tests.txt` and `05-helm-test.txt`.

Table 4.5 – Live checks of the four environments (level A; – = check not made)

Check	dev	test	staging	prod
Helm release status is deployed	PASS	PASS	PASS	PASS
Replicas ready	PASS (1/1)	PASS (1/1)	PASS (2/2)	PASS (3/3)
`APP_ENV` equals the expected value	PASS (development)	PASS (testing)	PASS (staging)	PASS (production)
`REPLICA_COUNT` in the ConfigMap equals the Deployment	PASS (1)	PASS (1)	PASS (2)	PASS (3)
`/health` returns ok	PASS	PASS	PASS	PASS
Production `DEBUG_MODE` is False	–	–	–	PASS
Production `SECRET_KEY` comes from a Secret	–	–	–	PASS
`helm test` of the release	PASS	PASS	PASS	PASS

A test that never fails proves little, so the suite was also checked from the other side: the 14 invalid inputs of Section 4.6 are rejected, the two failures of Section 4.8 are detected, and the availability experiment of Section 4.9 was designed to find a problem and found one. The automated cases cannot detect what they do not test: they do not measure traffic, scan the image or cover Minikube and Windows.

4.5 Reduction of Duplicated Configuration

To make the argument for the chart measurable, the effort of keeping *E* environments consistent is modelled by the number of lines that must be maintained (level C; lines are only a proxy for effort). Copied manifests need R lines for every environment; the chart has one fixed part, T lines of templates and V lines of default values, and every environment adds a values file of d lines:

$$L_{copy}(E) = E \cdot R \tag{4.1}$$

$$L_{chart}(E) = T + V + E \cdot d \tag{4.2}$$

The chart needs fewer lines than copies as soon as E exceeds the break-even number of Equation (4.3).

$$E^* = (T + V) / (R - d) \tag{4.3}$$

Counted from the repository, T = 186 lines (five templates including the test), V = 98 lines, R = 177.5 lines (the average rendered size, Table 3.8) and d = 31.75 lines (the average size of the environment files). The break-even number is E* = 1.95, or 2.59 if the schema (75 lines) and `Chart.yaml` (18 lines) count as fixed cost; the chart is therefore smaller than copies from the third environment onwards. For the four environments, copies would amount to 710 lines and the chart to 411 lines, which is 42 % fewer (29 % fewer with schema and metadata); a fifth environment would add 31.75 lines instead of 177.5, a saving of 82 % per additional environment (Figure 4.9). The model shows the size of the maintained text, not the time a change takes; its more important consequence is qualitative: a template is changed once, and the differences between the environments are visible as the few lines of a values file.

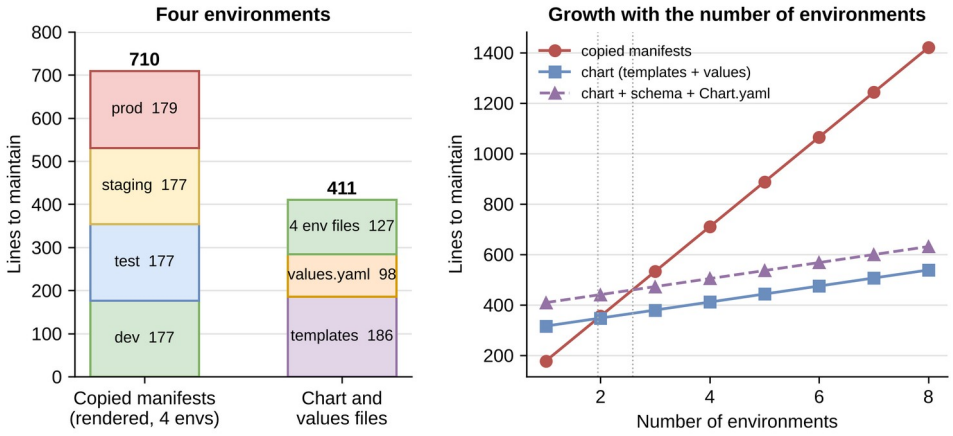


Figure 4.9 – Lines to maintain: copied manifests compared with the chart and values files (level C)

4.6 Schema Validation Results

To test the schema from the outside, 14 invalid inputs were each given to `helm lint ./helm/multi-env-app --strict --set <input>` (level B). All 14 were rejected with the exit status 1 and a message that names the offending key (Table 4.6); five of them are also pytest cases. The four environment files pass the same command (TC-04).

Table 4.6 – Invalid values rejected by values.schema.json

Input (--set)	Exit	Message of Helm
`environment=qa`	1	environment: environment must be one of the following: "development", "testing", "staging", "production"
`replicaCount=0`	1	replicaCount: Must be greater than or equal to 1
`replicaCount=11`	1	replicaCount: Must be less than or equal to 10
`config.LOG_LEVEL=VERBOSE`	1	config.LOG_LEVEL: config.LOG_LEVEL must be one of the following: "DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"
`config.DEBUG_MODE=maybe`	1	config.DEBUG_MODE: config.DEBUG_MODE must be one of the following: "true", "false"
`config.SECRET_KEY=abc`	1	config: Must not validate the schema (not)
`config.log_level=info`	1	config: Does not match pattern '^[A-Z][A-Z0-9_]*$'
`service.type=Foo`	1	service.type: service.type must be one of the following: "ClusterIP", "NodePort", "LoadBalancer"
`service.nodePort=80`	1	service.nodePort: Must be greater than or equal to 30000
`image.pullPolicy=Sometimes`	1	image.pullPolicy: image.pullPolicy must be one of the following: "Always", "IfNotPresent", "Never"
`containerPort=0`	1	containerPort: Must be greater than or equal to 1
`config.MAX_CONNECTIONS=abc`	1	config.MAX_CONNECTIONS: Does not match pattern '^[0-9]+$'
`config.CACHE_TTL_SECONDS=130` (number, not string)	1	config.CACHE_TTL_SECONDS: Invalid type. Expected: string, given: integer
`image.repository=""`	1	image.repository: String length must be greater than or equal to 1

The rejected inputs cover the classes of mistake that the studies of Chapter II report as frequent: an unknown name, numbers outside their range, values outside an enumeration, values with the wrong format or type, an empty required string, and a secret placed in the ConfigMap. Two observations are worth recording. First, `--set config.CACHE_TTL_SECONDS=130` is rejected because Helm reads 130 as an integer, whereas the schema and the ConfigMap require a string; the correct form is `--set-string`. Second, the message for a forbidden key (`Must not validate the schema (not)`) does not name the key, which makes this one rule harder to diagnose than the others. The cases were written by the authors of the schema, so they show that the rules do what they were written to do, not that the set of rules is complete.

4.7 Upgrade and Rollback

The release lifecycle was exercised on the development release with `scripts/upgrade-rollback-demo.sh dev` (level A). The upgrade set `app.version` to 1.1.0 and `LOG_LEVEL` to INFO with `--set`; the rollback returned to revision 1. Figure 4.10 shows the commands and the history, and Figure 4.11 the dashboards of the two later revisions.

```
bash - helm
$ helm upgrade myapp-dev ./helm/multi-env-app -n dev -f values-dev.yaml --set \
    app.version=1.1.0 --set config.LOG_LEVEL=INFO --atomic --wait --timeout 5m
Release "myapp-dev" has been upgraded. Happy Helming!
NAME: myapp-dev
LAST DEPLOYED: Tue Oct 20 18:12:08 2026
NAMESPACE: dev

$ helm history myapp-dev -n dev
REVISION  STATUS      CHART                APP VERSION  DESCRIPTION
1         superseded  multi-env-app-1.0.0  1.0.0        Install complete
2         deployed    multi-env-app-1.0.0  1.0.0        Upgrade complete

$ helm rollback myapp-dev 1 -n dev --wait --timeout 5m
Rollback was a success! Happy Helming!

$ helm history myapp-dev -n dev
REVISION  STATUS      CHART                APP VERSION  DESCRIPTION
1         superseded  multi-env-app-1.0.0  1.0.0        Install complete
2         superseded  multi-env-app-1.0.0  1.0.0        Upgrade complete
3         deployed    multi-env-app-1.0.0  1.0.0        Rollback to 1
```

Figure 4.10 – Upgrade, history, rollback and history of the development release (update times omitted)

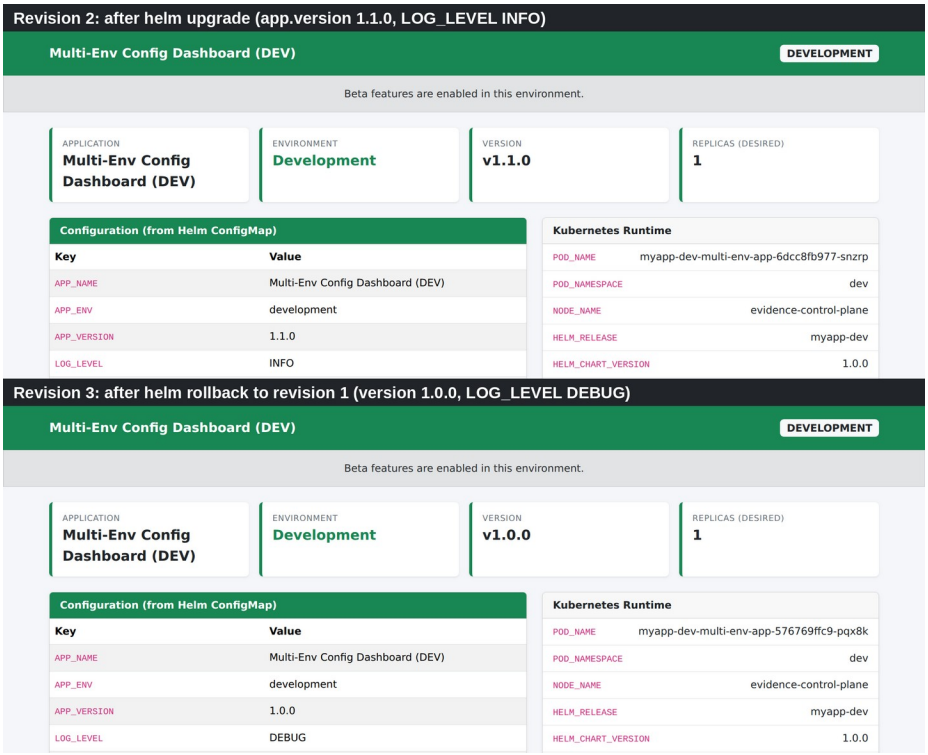


Figure 4.11 – Dashboard of the development release after the upgrade (revision 2) and after the rollback (revision 3); upper part of the page

Four points follow. (1) The upgrade replaced the Pod: the change of `LOG_LEVEL` and `app.version` changed the ConfigMap, therefore the checksum annotation and the Pod

template, so a new ReplicaSet and a new Pod were created without a manual restart. (2) The rollback did not delete revision 2; it created revision 3 with the description *Rollback to 1*, so the history stays a complete record. (3) The Pod of revision 3 carries the same template hash (`576769ffc9`) as the Pod of revision 1, which shows that the Deployment reused the ReplicaSet of revision 1. (4) The column APP VERSION of `helm history` stays at 1.0.0 for revision 2 because it shows the `appVersion` of the chart, not the value `app.version` that the dashboard displays.

4.8 Forced Failures

Two failures were provoked on purpose, both on the kind cluster (level A). First, the test release was upgraded with `--set image.tag=9.9.9`, an image that does not exist, with `--atomic --wait --timeout 60s` (Figure 4.12). The new Pod stayed in `ErrImagePull` and later `ImagePullBackOff`, while the old Pod was still *Running* and *Ready* (1/1) 35 s into the upgrade: this is the guarantee of `maxUnavailable: 0`, an old Pod is removed only after a new one is ready. After the timeout Helm reported *UPGRADE FAILED ... has been rolled back due to atomic being set*; the history shows revision 2 as *failed* and a new revision 3 *Rollback to 1* as *deployed*. `deploy.sh` always uses `--atomic` with a timeout of five minutes, so every deployment has this protection; the experiment also shows that a failed deployment is detected only after that time.

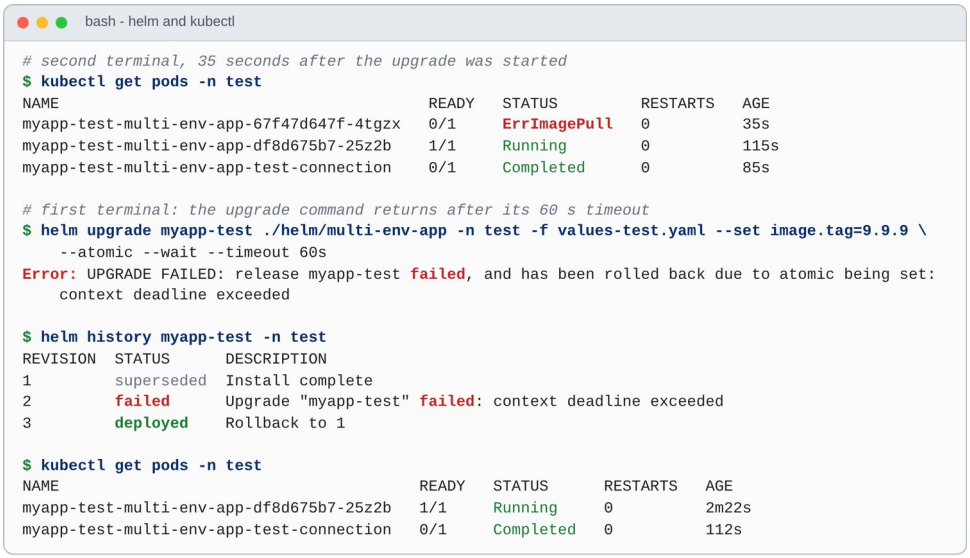


Figure 4.12 – Failed upgrade with `--atomic`: the old Pod keeps running and Helm restores revision 1 as revision 3

Second, the production values were installed as the release `demo-prod` in an empty namespace without the Secret `multi-env-app-secrets` (Figure 4.13). The command ended with `INSTALLATION FAILED: context deadline exceeded` after 40 s; the three Pods

were in the state `CreateContainerConfigError`, and the events name the cause: *secret "multi-env-app-secrets" not found*. Helm cannot detect this problem because the chart only refers to the Secret by name, and its message (a timeout) does not mention the Secret, so the diagnosis needs `kubectl get events`. This is why `deploy.sh` creates the Secret before it installs production. A check with the function `lookup` (which works only against a live cluster), a pre-install hook or an external secrets operator would detect the problem earlier (Chapter V). `--atomic` was not used in this experiment; the evidence script removed the release and the namespace afterwards.



Figure 4.13 – Production installed without its Secret (`helm install`, `kubectl get pods`, events)

4.9 Availability during Rolling Updates

The bounds Equation (3.1) and Equation (3.2) say how many Pods exist during an update, not whether a client notices it. A probe Pod (`busybox:1.36`) in the namespace of the release requested `/ready` from the Service with `wget` about nine times per second for 45 s and logged the second of every failed request. Eight seconds after the start, `helm upgrade --wait` changed `config.CACHE_TTL_SECONDS` (with `--set-string`), which changed the ConfigMap and the checksum annotation and started a rolling update. Five scenarios were run (Table 4.7): a control without an update; the strategy of the chart on two replicas (A, staging) and on one replica (B, test); the alternative `maxSurge: 0, maxUnavailable: 1` (C); and the chart strategy with a `preStop` hook that sleeps for 5 s (D). The hook was added to the live Deployment with `kubectl patch` and is **not** part of the delivered chart.

Table 4.7 – Failed requests during rolling updates (level A; totals over the runs of a scenario)

Scenario	Runs	Requests	Failed	Failure rate	Failed per run	Longest burst (s)
Z control: release unchanged (test, 1 replica)	1	424	0	0.00 %	0	none
A chart strategy, staging, 2 replicas	3	1,205	18	1.49 %	5–7	1–3
B chart strategy, test, 1 replica	3	1,247	15	1.20 %	3–7	2–4
C `maxSurge: 0`, `maxUnavailable: 1`, test, 1 replica	3	1,074	38	3.54 %	12–13	8–10
D chart strategy + `preStop` sleep 5 s (experiment only), test	3	1,285	0	0.00 %	0	none

A burst is a group of failed requests whose times differ by at most 3 s. Rollout times reported by Helm: A 15–16 s, B 8–9 s, C 2–3 s, D 8 s.

The control produced no failure in 424 requests, so the probe and the path through the Service are reliable and the failures of the other scenarios are caused by the rollouts. With the strategy of the chart, 18 of 1,205 requests failed in scenario A (1.49 %) and 15 of 1,247 in scenario B (1.20 %); the alternative lost 38 of 1,074 (3.54 %); with the `preStop` delay none of 1,285 failed (Figure 4.14).

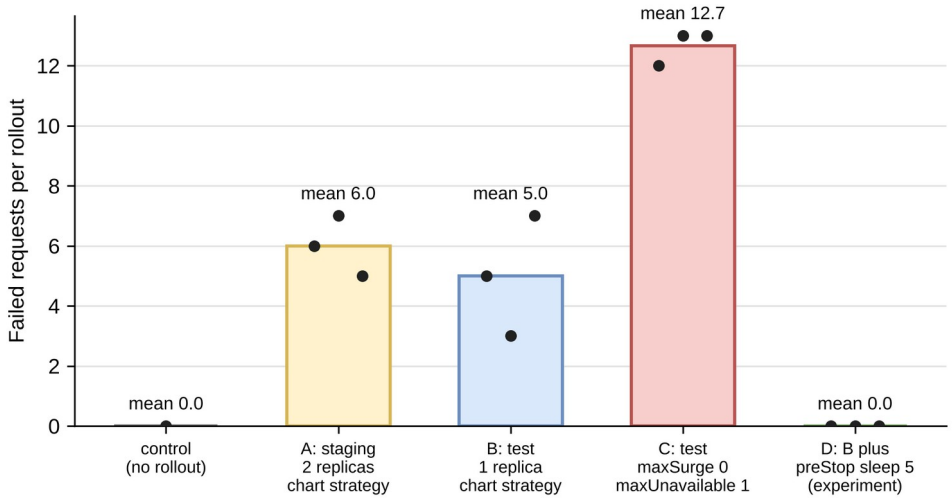


Figure 4.14 – Failed requests per rollout in the five scenarios (bars: mean; dots: individual runs)

1. **The chart strategy loses few requests, but not none.** Although the number of available Pods never fell below N, 1.2 to 1.5 % of the requests failed, in short bursts of 1 to 4 s just after the new Pod became ready, when the old Pod is shut down (*Connection refused*, *Connection reset by peer*, *download timed out*). A terminating Pod is removed from the endpoints of the Service while its container receives the termination signal, and for a short time the Service can still send a request to it [39]. With two replicas every run shows two bursts about 7 s apart, one for each replaced Pod.

2. **The alternative strategy is worse.** With one replica and `maxUnavailable: 1` the old Pod is removed first and the Service has no ready Pod until the new one passes its readiness probe: one burst of 8 to 10 s. Helm reported the rollout as finished after 2 to 3 s because `--wait` waits for the number of replicas minus `maxUnavailable` ready Pods [22, 40], which is zero here; with this strategy `--wait` and `--atomic` could not detect a broken revision, a second reason for `maxUnavailable: 0`.
3. **A preStop delay removed the failures in this experiment.** With `sleep 5` the old Pod keeps answering for 5 s after its deletion has started, which gives the endpoint update time to reach the Service. With zero failures in n = 1,285 requests the rule of three gives an upper bound of 0.23 % for the failure rate with 95 % confidence (level C).
4. **Consequence.** The strategy of the chart is the better of the two that were compared, but it does not give *zero* downtime for a client that is connected while an old Pod is removed; the documentation therefore says that the rollout *keeps full capacity*. A `preStop` hook, a `terminationGracePeriodSeconds` and a graceful shutdown of gunicorn are the first item of future work; they were not added to the chart because the finding was made after the chart had been tested, and the report describes the chart as delivered.

The experiment has limits: one node, a probe inside the cluster that uses the cluster address of the Service (not a NodePort or a load balancer), one client at about nine requests per second, three runs per scenario, and a failing `wget` slows the client down, so the numbers should be compared only within this experiment. A multi-node cluster may propagate endpoint changes more slowly, which would make the effect larger, not smaller. A first run was discarded because the command that waited for the probe Pod had a timeout without a unit and returned at once; the script was corrected and the whole experiment repeated, and only the second run is reported.

4.10 Image and Pipeline Results

Image. The image `multi-env-app:1.0.0` is 126 MB: the Debian base layer (78.8 MB) and the Python runtime layers (40.6 MB) account for 95 % of it, Flask and gunicorn add 6.39 MB, and the application code and the user add 27 kB (`docker history`, level A). Because the dependencies are installed before the code is copied, a change of the code rebuilds only the last two, very small layers; the build took 9 s in the validation workflow.

Pipeline. Figure 4.15 shows the timeline of the validation workflow for commit 9f0e6bc. The three independent jobs took 10 s, 17 s and 10 s and ran in parallel; the cluster job, which started when they had finished, took 143 s: 41 s to create the kind cluster, 40 s to deploy the four environments, 12 s for the live checks and 29 s for the upgrade and rollback. The whole run needed 164 s. All six runs of the workflow on the branch up to this commit succeeded; the workflow `capture-evidence.yml` (run 2) took 15 min 41 s, most of it for the 13 runs of the availability experiment.

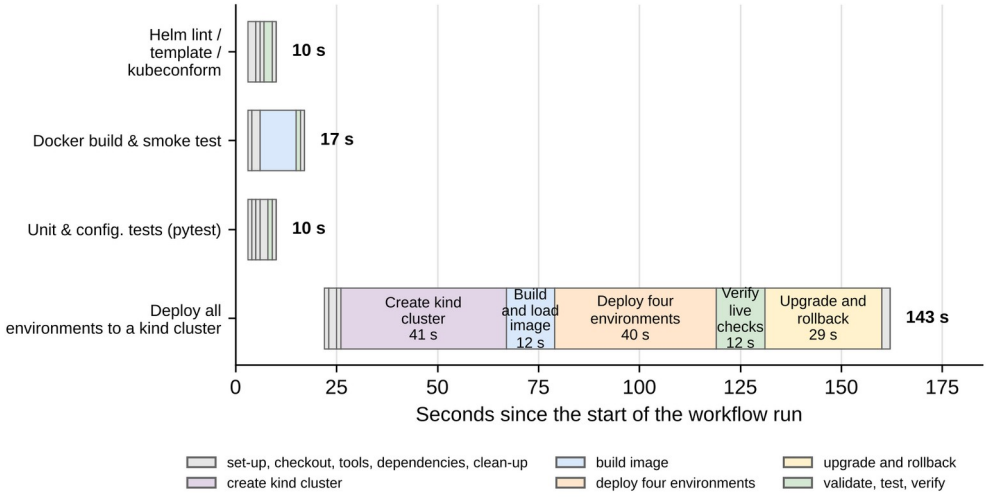


Figure 4.15 – Timeline of the four jobs of the validation workflow (run 6, commit 9f0e6bc)

4.11 Discussion

Table 4.8 compares the objectives of Section 1.4 with the results. Nine of the ten were achieved as stated; objective O6 was achieved on a kind cluster, and the Minikube path is supported by the scripts but was not executed (level D).

Table 4.8 – Objectives compared with the results

No.	Objective	Result	Evidence
O1	Flask, HTML, CSS and Bootstrap application	Achieved	Figure 4.4; 15 application tests
O2	Dashboard with name, environment, version and configuration	Achieved	Table 4.3: 56 of 56 values equal to the files
O3	Four environments	Achieved	Table 4.2; Figure 4.1
O4	Reusable chart with the required files	Achieved	Appendix A; lint, render and kubeconform valid
O5	Docker configuration	Achieved	Image of 126 MB; smoke test in CI; non-root user
O6	Kubernetes deployment on Minikube	Achieved on kind; Minikube not executed	Section 4.1; `minikube-setup.sh`
O7	Install, validation, upgrade and rollback	Achieved	TC-08, TC-09; Section 4.7
O8	GitHub Actions validation	Achieved	Figure 4.15; six successful runs

No.	Objective	Result	Evidence
O9	Test scripts for environment-specific configuration	Achieved	62 pytest cases; 26 live checks; 4 `helm test` runs
O10	No hard-coded secrets; consistent configuration	Achieved within the stated limits	Table 3.13; Section 4.6

1. **Separation of configuration works.** One set of templates and four values files of 28 to 43 lines produced four different environments; all 56 compared values of the running Pods equalled the files, and the chart is smaller than copied manifests from the third environment onwards.
2. **Validation before deployment is effective and cheap.** The schema rejected all 14 invalid inputs before any cluster was contacted; its weak point is the message for forbidden keys.
3. **Release history makes changes reversible.** The rollback restored version 1.0.0 and the level DEBUG as revision 3, and `--atomic` restored revision 1 after a failed upgrade while the old Pod kept serving.
4. **“Zero downtime” needs a measurement.** A rolling update with `maxUnavailable: 0` keeps capacity but lost 1.2 to 1.5 % of the requests in short bursts; a `preStop` delay removed these failures in three of three runs.
5. **Helm does not see a missing Secret**, and strict types cost a little convenience: numeric settings need `--set-string`.

The findings are consistent with the literature of Chapter II: misconfigurations are frequent and often consist of illegal values [2], which the schema turns into a check before deployment, and security studies of manifests [16] report missing resource limits and weak security contexts, which the chart avoids by default without claiming to remove all such misconfigurations. Threats to validity: one single-node kind cluster on one type of runner (Minikube and Windows not executed); a synthetic application without a real database or traffic; three runs per availability scenario with one client and no statistical test; line counts as a proxy for effort against a hypothetical baseline; tests and schema cases written by the authors of the code; and manifests checked against the schemas of Kubernetes 1.31.0 while the cluster ran v1.35.0 (the chart uses only the stable APIs `v1` and `apps/v1`).

CHAPTER 5
CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Conclusion

The project set out to show that one application can be deployed to development, testing, staging and production from a single Helm chart, without copying manifests, and that the result can be validated, tested, upgraded and reverted. The delivered solution consists of a Flask dashboard with Bootstrap, one container image of 126 MB, the chart `multi-env-app` with one file of default values, four environment files of 28 to 43 lines, a JSON Schema for the values, nine scripts, 62 automated tests and a GitHub Actions workflow of four jobs. On a kind cluster in GitHub Actions all four releases were deployed in 40 s; every one of the 56 compared settings of the running Pods equalled the values of the files; all 26 live checks, which include the four chart tests, passed; a rollback restored an earlier revision as a new revision; and `--atomic` restored a release after an upgrade to an image that does not exist. The schema rejected all 14 invalid inputs before any cluster was contacted, and the model of the effort shows that the chart is smaller than copied manifests from the third environment onwards.

The report also records two results that go against a simple story. A rolling update with `maxUnavailable: 0` keeps full capacity, but 1.5 % and 1.2 % of the requests failed in short bursts while old Pods were removed, whereas a `preStop` delay removed the failures in the experiment; “zero downtime” is therefore not claimed anywhere in the project. And Helm cannot detect that a Secret is missing, because the chart refers to it only by name. The report states for every result whether it was run on a cluster, run locally or derived, and it lists what was not executed, most importantly Minikube.

5.2 Summary of Contributions

1. A small, complete and reproducible reference for multi-environment configuration management with Helm that can be read in one sitting and run on a laptop cluster.
2. A typed interface for the values, enforced by a JSON Schema, that rejects unknown environments, wrong types, out-of-range numbers and secrets placed in the configuration.
3. A deployment workflow with `--atomic`, history and rollback, scripts for each step, a live test of every environment and a CI workflow that deploys all four environments to a real cluster on every change.

4. Measurements that go beyond a demonstration (two forced failures and a rolling-update experiment with a control, two strategies and a mitigation) and an explicit evidence level for every result.

5.3 Comparison with the Literature

Table 5.1 relates the observations of this report to the sources of Chapter II. The relation is one of consistency, not of replication: the project is a single small example, and the studies cited analyse hundreds or thousands of real repositories.

Table 5.1 – Observations of this project compared with the literature

Source	What the literature reports	What this project observed	Relation
Wiggins [1]	Configuration that varies between deploys belongs in the environment	All settings come from environment variables; one image runs in all four environments	Followed
Yin et al. [2]	Many misconfigurations are illegal parameter values	The schema rejected 14 of 14 invalid inputs before deployment	Consistent
Xu and Zhou [3]	Prevention, detection and diagnosis are separate tasks	Schema (prevention), lint, tests, CI (detection); diagnosis relied on events and logs	Partly addressed
Rahman et al. [14, 15]	Configuration code is defect-prone and needs testing	62 tests and four validation stages; the rollout failures were found by an experiment, not a test	Consistent
Rahman et al. [16]	Missing limits and weak security contexts are common	Limits in every environment, non-root user, capabilities dropped; no policy scanner	Addressed within limits
Zerouali et al. [13]	Public charts contain outdated and vulnerable components	Versions pinned, slim base image; no vulnerability scan was run	Not verified
Limoncelli; Humble and Farley; Forsgren et al. [4, 5, 19]	Desired state in version control; automated delivery pipeline	All files in Git; one workflow, six successful runs; no GitOps controller	Partly

5.4 Practical Considerations and Limitations

The project is a teaching example. Before a similar chart is used for a real application: use separate clusters where production requires them (namespaces separate names, not failure domains; only the values file and the cluster addressed by `helm` change); use an external secrets operator or sealed secrets with encryption at rest; measure the application under load and tune the requests and limits; place an Ingress with TLS and a disruption budget in front of the Service; promote the same chart version and image tag from environment to environment; and add metrics, logs and alerts, for which the probe of the availability experiment could become a permanent synthetic check.

The limitations of this report are: Minikube, Windows, multi-node and cloud clusters were not executed (level D, Section 4.1); the application is synthetic and stateless, without a database, real traffic or persistent storage; the chart contains no Ingress, TLS, autoscaling, disruption budget or network policy, and the production Secret is created outside the chart by

a script; the rollout of the chart is not free of request failures (Section 4.9) and the `preStop` hook that removed them is not part of the chart; the image was not scanned and its base image is selected by tag, not by digest; and the availability experiment is small (one single-node cluster, one client, three runs per scenario).

5.5 Future Enhancements

Figure 5.1 orders the enhancements by effort and dependency, and Table 5.2 gives the reason for each one, taken from the results of this report. The first two follow directly from the failures found in Section 4.8 and Section 4.9.

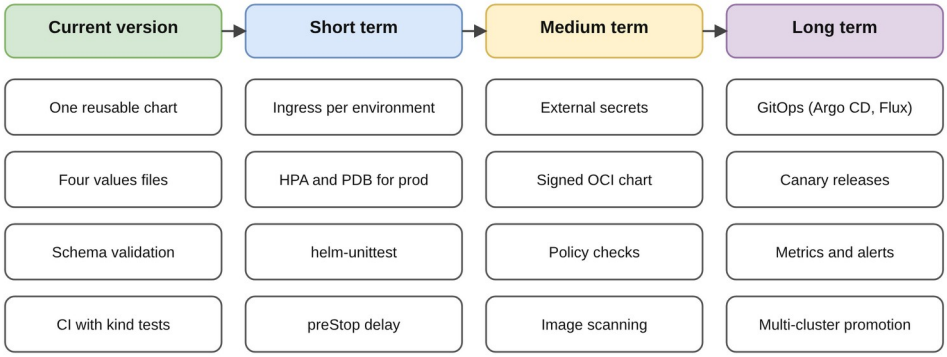


Figure 5.1 – Roadmap of future enhancements

Table 5.2 – Future enhancements and their reasons

Enhancement	Reason (evidence in this report)	Outline
Graceful shutdown: `preStop` delay, `terminationGracePeriodSeconds`, SIGTERM handling in gunicorn	1.2 to 1.5 % of the requests failed during rollouts; none with a 5 s delay (Section 4.9)	Add a `lifecycle.preStop` value to the chart; repeat the experiment with the delivered chart
Check or provide the Secret	Helm did not detect the missing Secret (Section 4.8)	A `lookup` check, a pre-install hook, or an external secrets operator
Ingress per environment with TLS; HPA and PDB for production	The dashboards are reached by NodePort only; three fixed replicas, no protection against node drains	One Ingress host per environment, cert-manager; `HorizontalPodAutoscaler` and `PodDisruptionBudget` in `values-prod.yaml`
Unit tests for templates; policy checks	Template logic and hardening are checked by pytest and hand-written assertions	`helm-unittest` suites; Conftest or Kyverno policies on the rendered manifests
Image scanning and a signed OCI chart	The image was not scanned; the base image is selected by tag	Trivy in CI; `helm push` to an OCI registry; signatures
GitOps controller, metrics and alerts, canary releases	Deployments are started by scripts and CI; failures were found by a probe written for this report	Argo CD or Flux reconcile the values files; Prometheus metrics and alert rules
Run on Minikube, Windows, a multi-node and a cloud cluster	These platforms are level D	Repeat the evidence workflow on each platform

None of these enhancements changes the central idea of the project: the application is described once, the differences between the environments are data, the data are validated before deployment, and every deployment is a numbered revision that can be reverted.

APPENDICES

Appendix A – Directory structure of the project

The line count of each file is given at the right. The files in the folder `docs/` (the diagram, the video and the evidence files) are not listed.

Code Snippet – Directory structure (git ls-files)

```
multi-env-helm-config/
├── .github/
│   └── workflows/
│       ├── capture-evidence.yml                    56 lines
│       └── helm-validation.yml                    117 lines
├── app/
│   ├── static/
│   │   └── css/
│   │       └── style.css                           43 lines
│   ├── templates/
│   │   └── index.html                             111 lines
│   ├── app.py                                      73 lines
│   ├── config.py                                  101 lines
│   └── requirements.txt                             2 lines
├── helm/
│   └── multi-env-app/
│       ├── templates/
│       │   ├── tests/
│       │   │   └── test-connection.yaml            27 lines
│       │   ├── _helpers.tpl                        48 lines
│       │   ├── configmap.yaml                      16 lines
│       │   ├── deployment.yaml                     77 lines
│       │   ├── NOTES.txt                           17 lines
│       │   └── service.yaml                        18 lines
│       ├── .helmignore                             10 lines
│       ├── Chart.yaml                              18 lines
│       ├── values-dev.yaml                         28 lines
│       ├── values-prod.yaml                        43 lines
│       ├── values-staging.yaml                     28 lines
│       ├── values-test.yaml                        28 lines
│       ├── values.schema.json                      75 lines
│       └── values.yaml                             98 lines
├── scripts/
│   ├── capture-evidence.sh                        238 lines
│   ├── cleanup.sh                                  12 lines
│   ├── common.sh                                   43 lines
│   ├── deploy.sh                                   49 lines
│   ├── docker-demo.sh                              63 lines
│   ├── minikube-setup.sh                           25 lines
│   ├── test-environments.sh                        68 lines
│   ├── upgrade-rollback-demo.sh                    50 lines
│   └── validate-helm.sh                            45 lines
├── tests/
│   ├── conftest.py                                  9 lines
│   ├── test_app.py                                 93 lines
│   └── test_helm_config.py                        230 lines
├── .dockerignore                                   13 lines
├── Dockerfile                                      31 lines
├── README.md                                      447 lines
└── requirements-dev.txt                             3 lines

docs/                                  diagram, video, evidence files
```

Appendix B – Application source code

Appendices B to F print complete files of the repository: a program wrote their lines, with the line numbers of the file, into the report, and the finished document was compared with the files. Files not printed are listed in Appendix G.

Code Snippet – app/app.py (complete, 73 lines)

```
"""Flask web application that displays its environment-specific configuration.

Endpoints:
    /            HTML dashboard (Bootstrap)
```

```
    /health       Liveness probe
    /ready        Readiness probe
    /api/config   JSON view of the (non-secret) configuration
"""

import logging
import os
from datetime import datetime, timezone

from flask import Flask, jsonify, render_template

from config import AppConfig

STARTED_AT = datetime.now(timezone.utc)


def create_app(environ=None):
    cfg = AppConfig(environ)

    app = Flask(__name__)
    app.config["SECRET_KEY"] = cfg.secret_key
    app.config["APP_CONFIG"] = cfg

    logging.basicConfig(
        level=getattr(logging, cfg.log_level, logging.INFO),
        format="%(asctime)s %(levelname)s [%(name)s] %(message)s",
    )
    log = logging.getLogger("multi-env-app")
    if not cfg.environment_is_valid:
        log.warning("APP_ENV is not one of the supported environments; defaulting to 'development'")
    log.info("Starting %s v%s in %s environment", cfg.app_name, cfg.app_version, cfg.environment)

    @app.route("/")
    def dashboard():
        return render_template(
            "index.html",
            cfg=cfg,
            settings=cfg.public_settings(),
            runtime=cfg.runtime_info(),
            started_at=STARTED_AT.strftime("%Y-%m-%d %H:%M:%S UTC"),
        )

    @app.route("/health")
    def health():
        return jsonify(status="ok"), 200

    @app.route("/ready")
    def ready():
        return jsonify(status="ready", environment=cfg.environment), 200

    @app.route("/api/config")
    def api_config():
        return jsonify(
            application=cfg.app_name,
            environment=cfg.environment,
            version=cfg.app_version,
            settings=cfg.public_settings(),
            runtime=cfg.runtime_info(),
        )

    return app


app = create_app()


if __name__ == "__main__":
    # Local development server only. In containers the app runs under gunicorn.
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", "5000")),
   debug=app.config["APP_CONFIG"].debug)
```

Code Snippet – app/config.py (complete, 101 lines)

```
"""Runtime configuration for the Multi-Environment Config Dashboard.

All values are read from environment variables. In Kubernetes these
variables are injected from the Helm-generated ConfigMap (non-sensitive
settings) and, optionally, from a pre-created Kubernetes Secret.
Nothing sensitive is hardcoded here.
"""

import os
import secrets

VALID_ENVIRONMENTS = ("development", "testing", "staging", "production")

# Bootstrap colour used for the environment badge / banner on the dashboard.
ENVIRONMENT_THEMES = {
    "development": "success",
    "testing": "info",
```

```
        "staging": "warning",
        "production": "danger",
    }

    # Keys whose values must never be displayed or returned by the API.
    SENSITIVE_KEYS = ("SECRET_KEY",)


    def _as_bool(value, default=False):
        if value is None:
            return default
        return str(value).strip().lower() in ("1", "true", "yes", "on")


    def _as_int(value, default):
        try:
            return int(value)
        except (TypeError, ValueError):
            return default


    class AppConfig:
        """Snapshot of the application configuration taken from the environment."""

        def __init__(self, environ=None):
            env = os.environ if environ is None else environ

            self.app_name = env.get("APP_NAME", "Multi-Env Config Dashboard")
            raw_env = env.get("APP_ENV", "development").strip().lower()
            self.environment_is_valid = raw_env in VALID_ENVIRONMENTS
            self.environment = raw_env if self.environment_is_valid else "development"
            self.app_version = env.get("APP_VERSION", "1.0.0")

            self.log_level = env.get("LOG_LEVEL", "INFO").upper()
            self.debug = _as_bool(env.get("DEBUG_MODE"), default=False)
            self.feature_beta_banner = _as_bool(env.get("FEATURE_BETA_BANNER"))
            self.max_connections = _as_int(env.get("MAX_CONNECTIONS"), 10)
            self.cache_ttl_seconds = _as_int(env.get("CACHE_TTL_SECONDS"), 60)
            self.database_host = env.get("DATABASE_HOST", "localhost")
            self.api_base_url = env.get("API_BASE_URL", "http://localhost:5000/api")
            self.replica_count = env.get("REPLICA_COUNT", "1")

            # Kubernetes downward-API values (present only inside a pod).
            self.pod_name = env.get("POD_NAME", "local")
            self.pod_namespace = env.get("POD_NAMESPACE", "local")
            self.node_name = env.get("NODE_NAME", "local")
            self.helm_release = env.get("HELM_RELEASE", "not-deployed-with-helm")
            self.chart_version = env.get("HELM_CHART_VERSION", "n/a")

            # Secret key: supplied via a Kubernetes Secret, or generated per
            # process so that no secret is ever committed to source control.
            provided_secret = env.get("SECRET_KEY")
            self.secret_key_source = "kubernetes-secret" if provided_secret else "auto-generated"
            self.secret_key = provided_secret or secrets.token_hex(32)

        @property
        def theme(self):
            return ENVIRONMENT_THEMES[self.environment]

        def public_settings(self):
            """Settings that are safe to show on the dashboard / JSON API."""
            return {
                "APP_NAME": self.app_name,
                "APP_ENV": self.environment,
                "APP_VERSION": self.app_version,
                "LOG_LEVEL": self.log_level,
                "DEBUG_MODE": self.debug,
                "FEATURE_BETA_BANNER": self.feature_beta_banner,
                "MAX_CONNECTIONS": self.max_connections,
                "CACHE_TTL_SECONDS": self.cache_ttl_seconds,
                "DATABASE_HOST": self.database_host,
                "API_BASE_URL": self.api_base_url,
                "REPLICA_COUNT": self.replica_count,
                "SECRET_KEY": "******** ({})".format(self.secret_key_source),
            }

        def runtime_info(self):
            return {
                "POD_NAME": self.pod_name,
                "POD_NAMESPACE": self.pod_namespace,
                "NODE_NAME": self.node_name,
                "HELM_RELEASE": self.helm_release,
                "HELM_CHART_VERSION": self.chart_version,
            }
```

Code Snippet – app/templates/index.html (complete, 111 lines)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{{ cfg.app_name }} - {{ cfg.environment|capitalize }}</title>
  <link rel="stylesheet"
   href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body class="env-{{ cfg.environment }}">
  <nav class="navbar navbar-dark bg-{{ cfg.theme }} env-banner">
    <div class="container">
      <span class="navbar-brand fw-bold">{{ cfg.app_name }}</span>
      <span class="badge bg-light text-dark fs-6" id="env-badge">{{ cfg.environment|upper }}</span>
    </div>
  </nav>

  {% if cfg.feature_beta_banner %}
  <div class="alert alert-secondary rounded-0 mb-0 text-center" role="alert">
    Beta features are enabled in this environment.
  </div>
  {% endif %}

  <main class="container py-4">
    <div class="row g-3 mb-4">
      <div class="col-md-3 col-sm-6">
        <div class="card stat-card border-{{ cfg.theme }} h-100">
          <div class="card-body">
            <div class="stat-label">Application</div>
            <div class="stat-value" id="app-name">{{ cfg.app_name }}</div>
          </div>
        </div>
      </div>
      <div class="col-md-3 col-sm-6">
        <div class="card stat-card border-{{ cfg.theme }} h-100">
          <div class="card-body">
            <div class="stat-label">Environment</div>
            <div class="stat-value text-{{ cfg.theme }}" id="app-env">{{ cfg.environment|
   capitalize }}</div>
          </div>
        </div>
      </div>
      <div class="col-md-3 col-sm-6">
        <div class="card stat-card border-{{ cfg.theme }} h-100">
          <div class="card-body">
            <div class="stat-label">Version</div>
            <div class="stat-value" id="app-version">v{{ cfg.app_version }}</div>
          </div>
        </div>
      </div>
      <div class="col-md-3 col-sm-6">
        <div class="card stat-card border-{{ cfg.theme }} h-100">
          <div class="card-body">
            <div class="stat-label">Replicas (desired)</div>
            <div class="stat-value">{{ cfg.replica_count }}</div>
          </div>
        </div>
      </div>
    </div>

    <div class="row g-4">
      <div class="col-lg-7">
        <div class="card shadow-sm">
          <div class="card-header bg-{{ cfg.theme }} text-white fw-semibold">
            Configuration (from Helm ConfigMap)
          </div>
          <div class="table-responsive">
            <table class="table table-striped mb-0 config-table">
              <thead><tr><th>Key</th><th>Value</th></tr></thead>
              <tbody>
              {% for key, value in settings.items() %}
                <tr><td><code>{{ key }}</code></td><td>{{ value }}</td></tr>
              {% endfor %}
              </tbody>
            </table>
          </div>
        </div>
      </div>

      <div class="col-lg-5">
        <div class="card shadow-sm mb-4">
          <div class="card-header fw-semibold">Kubernetes Runtime</div>
          <ul class="list-group list-group-flush">
            {% for key, value in runtime.items() %}
            <li class="list-group-item d-flex justify-content-between">
```

```
                <span><code>{{ key }}</code></span><span class="text-break text-end">{{ value }}</span>
              </li>
              {% endfor %}
              <li class="list-group-item d-flex justify-content-between">
                <span><code>STARTED_AT</code></span><span>{{ started_at }}</span>
              </li>
            </ul>
          </div>

          <div class="card shadow-sm">
            <div class="card-header fw-semibold">Endpoints</div>
            <ul class="list-group list-group-flush">
              <li class="list-group-item"><a href="/api/config">/api/config</a> - configuration as
      JSON</li>
              <li class="list-group-item"><a href="/health">/health</a> - liveness probe</li>
              <li class="list-group-item"><a href="/ready">/ready</a> - readiness probe</li>
            </ul>
          </div>
        </div>
      </div>
    </main>

    <footer class="text-center text-muted small py-3 border-top">
      Multi-Environment Configuration Management with Helm &middot;
      Arun Kumar T &amp;amp; Priyan M &middot; Dept. of AI &amp;amp; DS, AAA College of Engineering and
     Technology, Sivakasi
    </footer>
  </body>
  </html>
```

Code Snippet – app/static/css/style.css (complete, 43 lines)

```
/* Custom styles layered on top of Bootstrap 5. */

body {
  background-color: #f4f6f9;
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
}

.env-banner {
  padding-top: 0.9rem;
  padding-bottom: 0.9rem;
}

.stat-card {
  border-width: 0 0 0 5px;
  border-style: solid;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
}

.stat-label {
  font-size: 0.8rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: #6c757d;
}

.stat-value {
  font-size: 1.35rem;
  font-weight: 600;
  word-break: break-word;
}

.config-table td:first-child {
  width: 45%;
}

/* Production gets a subtle warning stripe so it is never mistaken for dev. */
body.env-production .env-banner {
  background-image: repeating-linear-gradient(
    45deg,
    rgba(255, 255, 255, 0.06) 0 12px,
    transparent 12px 24px
  );
}
```

Code Snippet – app/requirements.txt (complete, 2 lines)

```
Flask==3.0.3
gunicorn==23.0.0
```

Appendix C – Container files

Code Snippet – Dockerfile (complete, 31 lines)

```
# ------------------------------------------------------------
# Multi-Environment Config Dashboard - container image
# The same image is promoted through dev -> test -> staging -> prod;
# only the Helm values (ConfigMap) change between environments.
# ------------------------------------------------------------
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PORT=5000

WORKDIR /app

# Install dependencies first to make good use of the layer cache.
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ .

# Run as an unprivileged user (UID matches the Helm securityContext).
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin appuser \
    && chown -R appuser /app
USER 10001

EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD python -c "import urllib.request,sys; sys.exit(0 if
   urllib.request.urlopen('http://127.0.0.1:5000/health').status == 200 else 1)"

CMD ["sh", "-c", "exec gunicorn --bind 0.0.0.0:${PORT} --workers 2 --access-logfile - app:app"]
```

Code Snippet – .dockerignore (complete, 13 lines)

```
.git
.github
helm
docs
scripts
tests
dist
**/__pycache__
**/*.pyc
.pytest_cache
.venv
venv
*.md
```

Appendix D – Helm chart

The chart is listed in the order in which Helm reads it: metadata, defaults, the environment files of development and production (the other two have the same structure), the schema, the helpers and the templates.

Code Snippet – helm/multi-env-app/Chart.yaml (complete, 18 lines)

```
apiVersion: v2
name: multi-env-app
description: >-
  Reusable Helm chart that deploys the Multi-Environment Config Dashboard
  (Flask) to Development, Testing, Staging and Production using
  environment-specific values files.
type: application
# Chart version - bump whenever the chart templates change.
version: 1.0.0
# Application version - used as the default image tag and APP_VERSION.
appVersion: "1.0.0"
keywords:
  - flask
```

```
  - multi-environment
  - configuration-management
maintainers:
  - name: Arun Kumar T
  - name: Priyan M
```

Code Snippet – helm/multi-env-app/values.yaml (complete, 98 lines)

```
# -----------------------------------------------------------------------------
# Default (base) values for the multi-env-app chart.
# Environment files (values-dev.yaml, values-test.yaml, values-staging.yaml,
# values-prod.yaml) override only what differs from these defaults:
#
#   helm install myapp-dev ./helm/multi-env-app -f ./helm/multi-env-app/values-dev.yaml
#
# Do NOT put passwords, tokens or keys here. Sensitive values belong in a
# Kubernetes Secret that is referenced through `existingSecret`.
# -----------------------------------------------------------------------------

# One of: development, testing, staging, production
environment: development

replicaCount: 1

image:
  repository: multi-env-app
  # Defaults to .Chart.AppVersion when empty.
  tag: ""
  pullPolicy: IfNotPresent

nameOverride: ""
fullnameOverride: ""

app:
  name: "Multi-Env Config Dashboard"
  # Version shown on the dashboard. Defaults to the image tag, then appVersion.
  version: ""

# Non-sensitive settings. Every key becomes an environment variable via the
# ConfigMap (templates/configmap.yaml). Values are always rendered as strings.
config:
  LOG_LEVEL: "INFO"
  DEBUG_MODE: "false"
  FEATURE_BETA_BANNER: "false"
  MAX_CONNECTIONS: "10"
  CACHE_TTL_SECONDS: "60"
  DATABASE_HOST: "localhost"
  API_BASE_URL: "http://localhost:5000/api"

# Name of a pre-existing Secret whose keys are exposed as env vars
# (for example SECRET_KEY). Leave empty to let the app generate a random key.
#   kubectl create secret generic myapp-secrets -n prod --from-literal=SECRET_KEY=$(openssl rand -hex
    32)
existingSecret: ""

containerPort: 5000

service:
  type: NodePort
  port: 80
  # Fixed NodePort (30000-32767) so each environment has a predictable URL.
  # Leave empty to let Kubernetes choose one.
  nodePort: ""

strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0

resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 200m
    memory: 128Mi

livenessProbe:
  httpGet:
    path: /health
    port: http
  initialDelaySeconds: 5
  periodSeconds: 10

readinessProbe:
  httpGet:
    path: /ready
```

```
    port: http
  initialDelaySeconds: 3
  periodSeconds: 5

podAnnotations: {}

podSecurityContext:
  runAsNonRoot: true
  runAsUser: 10001
  fsGroup: 10001

securityContext:
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]

# Image used by `helm test` (templates/tests/test-connection.yaml).
tests:
  image: busybox:1.36
```

Code Snippet – helm/multi-env-app/values-dev.yaml (complete, 28 lines)

```
# Development: fast feedback, verbose logging, single replica, beta features on.
environment: development

replicaCount: 1

app:
  name: "Multi-Env Config Dashboard (DEV)"

config:
  LOG_LEVEL: "DEBUG"
  DEBUG_MODE: "true"
  FEATURE_BETA_BANNER: "true"
  MAX_CONNECTIONS: "5"
  CACHE_TTL_SECONDS: "10"
  DATABASE_HOST: "postgres.dev.svc.cluster.local"
  API_BASE_URL: "http://dev.multi-env-app.local/api"

service:
  type: NodePort
  nodePort: 30081

resources:
  requests:
    cpu: 25m
    memory: 64Mi
  limits:
    cpu: 100m
    memory: 128Mi
```

Code Snippet – helm/multi-env-app/values-prod.yaml (complete, 43 lines)

```
# Production: high availability, minimal logging, debug disabled.
# Sensitive values (SECRET_KEY) come from a pre-created Kubernetes Secret.
environment: production

replicaCount: 3

image:
  pullPolicy: IfNotPresent

app:
  name: "Multi-Env Config Dashboard"

config:
  LOG_LEVEL: "WARNING"
  DEBUG_MODE: "false"
  FEATURE_BETA_BANNER: "false"
  MAX_CONNECTIONS: "200"
  CACHE_TTL_SECONDS: "300"
  DATABASE_HOST: "postgres.prod.svc.cluster.local"
  API_BASE_URL: "https://multi-env-app.example.com/api"

# Create before installing (see README):
#   kubectl create secret generic multi-env-app-secrets -n prod \
#     --from-literal=SECRET_KEY="$(openssl rand -hex 32)"
existingSecret: "multi-env-app-secrets"

service:
  type: NodePort
  nodePort: 30084

strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
```

```
      maxUnavailable: 0

  resources:
    requests:
      cpu: 100m
      memory: 128Mi
    limits:
      cpu: 500m
      memory: 256Mi
```

Code Snippet – helm/multi-env-app/values.schema.json (complete, 75 lines)

```
{
  "$schema": "https://json-schema.org/draft-07/schema#",
  "title": "multi-env-app values",
  "type": "object",
  "required": ["environment", "replicaCount", "image", "app", "config", "service"],
  "properties": {
    "environment": {
      "type": "string",
      "enum": ["development", "testing", "staging", "production"],
      "description": "Target environment"
    },
    "replicaCount": {
      "type": "integer",
      "minimum": 1,
      "maximum": 10
    },
    "image": {
      "type": "object",
      "required": ["repository", "pullPolicy"],
      "properties": {
        "repository": { "type": "string", "minLength": 1 },
        "tag": { "type": "string" },
        "pullPolicy": { "type": "string", "enum": ["Always", "IfNotPresent", "Never"] }
      }
    },
    "app": {
      "type": "object",
      "required": ["name"],
      "properties": {
        "name": { "type": "string", "minLength": 1 },
        "version": { "type": "string" }
      }
    },
    "config": {
      "type": "object",
      "required": ["LOG_LEVEL", "DEBUG_MODE"],
      "properties": {
        "LOG_LEVEL": { "type": "string", "enum": ["DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"] },
        "DEBUG_MODE": { "type": "string", "enum": ["true", "false"] },
        "FEATURE_BETA_BANNER": { "type": "string", "enum": ["true", "false"] },
        "MAX_CONNECTIONS": { "type": "string", "pattern": "^[0-9]+$" },
        "CACHE_TTL_SECONDS": { "type": "string", "pattern": "^[0-9]+$" }
      },
      "propertyNames": { "pattern": "^[A-Z][A-Z0-9_]*$" },
      "not": {
        "anyOf": [
          { "required": ["SECRET_KEY"] },
          { "required": ["PASSWORD"] },
          { "required": ["DB_PASSWORD"] },
          { "required": ["API_TOKEN"] }
        ]
      }
    },
    "existingSecret": { "type": "string" },
    "containerPort": { "type": "integer", "minimum": 1, "maximum": 65535 },
    "service": {
      "type": "object",
      "required": ["type", "port"],
      "properties": {
        "type": { "type": "string", "enum": ["ClusterIP", "NodePort", "LoadBalancer"] },
        "port": { "type": "integer", "minimum": 1, "maximum": 65535 },
        "nodePort": {
          "oneOf": [
            { "type": "string", "maxLength": 0 },
            { "type": "integer", "minimum": 30000, "maximum": 32767 }
          ]
        }
      }
    },
    "tests": {
```

```
      "type": "object",
      "properties": { "image": { "type": "string", "minLength": 1 } }
    }
  }
}
```

Code Snippet – helm/multi-env-app/templates/_helpers.tpl (complete, 48 lines)

```
{{/* Chart name, overridable with .Values.nameOverride. */}}
{{- define "multi-env-app.name" -}}
{{- default .Chart.Name .Values.nameOverride | trunc 63 | trimSuffix "-" }}
{{- end }}

{{/*
Fully qualified app name. Uses the release name so several environments
can live side by side (myapp-dev, myapp-test, ...).
*/}}
{{- define "multi-env-app.fullname" -}}
{{- if .Values.fullnameOverride }}
{{- .Values.fullnameOverride | trunc 63 | trimSuffix "-" }}
{{- else }}
{{- $name := default .Chart.Name .Values.nameOverride }}
{{- if contains $name .Release.Name }}
{{- .Release.Name | trunc 63 | trimSuffix "-" }}
{{- else }}
{{- printf "%s-%s" .Release.Name $name | trunc 63 | trimSuffix "-" }}
{{- end }}
{{- end }}
{{- end }}

{{- define "multi-env-app.chart" -}}
{{- printf "%s-%s" .Chart.Name .Chart.Version | replace "+" "_" | trunc 63 | trimSuffix "-" }}
{{- end }}

{{/* Image tag: explicit value or the chart's appVersion. */}}
{{- define "multi-env-app.imageTag" -}}
{{- .Values.image.tag | default .Chart.AppVersion }}
{{- end }}

{{/* Application version shown on the dashboard. */}}
{{- define "multi-env-app.appVersion" -}}
{{- .Values.app.version | default (include "multi-env-app.imageTag" .) }}
{{- end }}

{{- define "multi-env-app.selectorLabels" -}}
app.kubernetes.io/name: {{ include "multi-env-app.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}

{{- define "multi-env-app.labels" -}}
helm.sh/chart: {{ include "multi-env-app.chart" . }}
{{ include "multi-env-app.selectorLabels" . }}
app.kubernetes.io/version: {{ include "multi-env-app.appVersion" . | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
app.kubernetes.io/environment: {{ .Values.environment }}
{{- end }}
```

Code Snippet – helm/multi-env-app/templates/configmap.yaml (complete, 16 lines)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "multi-env-app.fullname" . }}-config
  labels:
    {{- include "multi-env-app.labels" . | nindent 4 }}
data:
  APP_NAME: {{ .Values.app.name | quote }}
  APP_ENV: {{ .Values.environment | quote }}
  APP_VERSION: {{ include "multi-env-app.appVersion" . | quote }}
  REPLICA_COUNT: {{ .Values.replicaCount | quote }}
  HELM_RELEASE: {{ .Release.Name | quote }}
  HELM_CHART_VERSION: {{ .Chart.Version | quote }}
  {{- range $key, $value := .Values.config }}
  {{ $key }}: {{ $value | toString | quote }}
  {{- end }}
```

Code Snippet – helm/multi-env-app/templates/deployment.yaml (complete, 77 lines)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "multi-env-app.fullname" . }}
  labels:
```

```
    {{- include "multi-env-app.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  revisionHistoryLimit: 5
  {{- with .Values.strategy }}
  strategy:
    {{- toYaml . | nindent 4 }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "multi-env-app.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "multi-env-app.labels" . | nindent 8 }}
      annotations:
        # Roll the pods automatically whenever the ConfigMap content changes.
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
        {{- with .Values.podAnnotations }}
        {{- toYaml . | nindent 8 }}
        {{- end }}
    spec:
      {{- with .Values.podSecurityContext }}
      securityContext:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ include "multi-env-app.imageTag" . }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          {{- with .Values.securityContext }}
          securityContext:
            {{- toYaml . | nindent 12 }}
          {{- end }}
          ports:
            - name: http
              containerPort: {{ .Values.containerPort }}
              protocol: TCP
          envFrom:
            - configMapRef:
                name: {{ include "multi-env-app.fullname" . }}-config
            {{- if .Values.existingSecret }}
            - secretRef:
                name: {{ .Values.existingSecret }}
            {{- end }}
          env:
            - name: PORT
              value: {{ .Values.containerPort | quote }}
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
            - name: POD_NAMESPACE
              valueFrom:
                fieldRef:
                  fieldPath: metadata.namespace
            - name: NODE_NAME
              valueFrom:
                fieldRef:
                  fieldPath: spec.nodeName
          {{- with .Values.livenessProbe }}
          livenessProbe:
            {{- toYaml . | nindent 12 }}
          {{- end }}
          {{- with .Values.readinessProbe }}
          readinessProbe:
            {{- toYaml . | nindent 12 }}
          {{- end }}
          {{- with .Values.resources }}
          resources:
            {{- toYaml . | nindent 12 }}
          {{- end }}
```

Code Snippet – helm/multi-env-app/templates/service.yaml (complete, 18 lines)

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "multi-env-app.fullname" . }}
  labels:
    {{- include "multi-env-app.labels" . | nindent 4 }}
spec:
  type: {{ .Values.service.type }}
  selector:
    {{- include "multi-env-app.selectorLabels" . | nindent 4 }}
```

```
  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: http
      protocol: TCP
      {{- if and (eq .Values.service.type "NodePort") .Values.service.nodePort }}
      nodePort: {{ .Values.service.nodePort }}
      {{- end }}
```

Code Snippet – helm/multi-env-app/templates/tests/test-connection.yaml (complete, 27 lines)

```
# Run with: helm test <release> -n <namespace>
# Checks that the Service answers and that the pod reports the expected environment.
apiVersion: v1
kind: Pod
metadata:
  name: {{ include "multi-env-app.fullname" . }}-test-connection
  labels:
    {{- include "multi-env-app.labels" . | nindent 4 }}
  annotations:
    "helm.sh/hook": test
    "helm.sh/hook-delete-policy": before-hook-creation
spec:
  restartPolicy: Never
  containers:
    - name: check-environment
      image: {{ .Values.tests.image }}
      imagePullPolicy: IfNotPresent
      command: ["sh", "-c"]
      args:
        - |
          set -e
          URL="http://{{ include "multi-env-app.fullname" . }}:{{ .Values.service.port }}"
          wget -qO- "$URL/health"
          BODY="$(wget -qO- "$URL/api/config")"
          echo "$BODY"
          echo "$BODY" | grep -Eq '"environment": ?"{{ .Values.environment }}"'
          echo "OK: service reachable and environment is {{ .Values.environment }}"
```

Appendix E – Scripts

Code Snippet – scripts/common.sh (complete, 43 lines)

```
#!/usr/bin/env bash
# Shared settings for all scripts. Source this file; do not execute it.

PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
CHART_DIR="${PROJECT_ROOT}/helm/multi-env-app"
IMAGE_REPO="${IMAGE_REPO:-multi-env-app}"
IMAGE_TAG="${IMAGE_TAG:-$(awk '/^appVersion:/ {gsub(/"/, "", $2); print $2}'
   "${CHART_DIR}/Chart.yaml")}"
RELEASE_PREFIX="${RELEASE_PREFIX:-myapp}"
SECRET_NAME="multi-env-app-secrets"
ALL_ENVS=(dev test staging prod)

# Environment short name -> namespace / values file / expected APP_ENV / NodePort.
namespace_for()  { echo "$1"; }
values_for()     { echo "${CHART_DIR}/values-$1.yaml"; }
release_for()    { echo "${RELEASE_PREFIX}-$1"; }
app_env_for() {
  case "$1" in
    dev) echo development ;; test) echo testing ;;
    staging) echo staging ;; prod) echo production ;;
    *) return 1 ;;
  esac
}
node_port_for() {
  case "$1" in
    dev) echo 30081 ;; test) echo 30082 ;; staging) echo 30083 ;; prod) echo 30084 ;;
    *) return 1 ;;
  esac
}

require_env() {
  if ! app_env_for "${1:-}" >/dev/null; then
    echo "ERROR: environment must be one of: ${ALL_ENVS[*]} (got '${1:-}')" >&2
    exit 1
  fi
}

require_cmd() {
```

```
  for cmd in "$@"; do
    command -v "$cmd" >/dev/null 2>&1 || { echo "ERROR: '$cmd' is not installed or not on PATH" >&2; exit 1; }
  done
}

log() { printf '\n\033[1;34m==> %s\033[0m\n' "$*"; }
```

Code Snippet – scripts/deploy.sh (complete, 49 lines)

```
#!/usr/bin/env bash
# Install or upgrade the chart for one environment (or all of them).
#
# Usage: ./scripts/deploy.sh <dev|test|staging|prod|all> [extra helm args...]
# Example: ./scripts/deploy.sh staging --set replicaCount=3
set -euo pipefail
source "$(dirname "$0")/common.sh"
require_cmd kubectl helm

TARGET="${1:-}"
shift || true

ensure_prod_secret() {
  local ns="$1"
  if ! kubectl get secret "${SECRET_NAME}" -n "$ns" >/dev/null 2>&1; then
    log "Creating Secret ${SECRET_NAME} in ${ns} with a random SECRET_KEY"
    # Generated at deploy time - never stored in Git.
    local key
    key="$(head -c 32 /dev/urandom | od -An -tx1 | tr -d ' \n')"
    kubectl create secret generic "${SECRET_NAME}" -n "$ns" --from-literal=SECRET_KEY="${key}"
  fi
}

deploy_one() {
  local env="$1"; shift
  require_env "$env"
  local ns release values
  ns="$(namespace_for "$env")"
  release="$(release_for "$env")"
  values="$(values_for "$env")"

  log "Deploying ${release} to namespace ${ns} using $(basename "$values")"
  kubectl create namespace "$ns" --dry-run=client -o yaml | kubectl apply -f - >/dev/null
  [[ "$env" == "prod" ]] && ensure_prod_secret "$ns"

  helm upgrade --install "$release" "$CHART_DIR" \
    --namespace "$ns" \
    -f "$values" \
    --atomic --wait --timeout 5m \
    "$@"

  kubectl get deploy,svc,configmap,pods -n "$ns" -l "app.kubernetes.io/instance=${release}"
}

if [[ "$TARGET" == "all" ]]; then
  for env in "${ALL_ENVS[@]}"; do deploy_one "$env" "$@"; done
else
  deploy_one "$TARGET" "$@"
fi
```

Code Snippet – scripts/validate-helm.sh (complete, 45 lines)

```
#!/usr/bin/env bash
# Validate the Helm chart for every environment WITHOUT touching a cluster:
#   1. helm lint --strict (also enforces values.schema.json)
#   2. helm template (render) for each values file
#   3. kubeconform schema validation of the rendered manifests (if installed)
#
# Usage: ./scripts/validate-helm.sh
set -euo pipefail
source "$(dirname "$0")/common.sh"
require_cmd helm

OUT_DIR="${PROJECT_ROOT}/dist/rendered"
mkdir -p "$OUT_DIR"
failures=0

for env in "${ALL_ENVS[@]}"; do
  values="$(values_for "$env")"
  log "[$env] helm lint --strict"
  helm lint "$CHART_DIR" --strict -f "$values" || failures=$((failures + 1))

  log "[$env] helm template"
  out="${OUT_DIR}/${env}.yaml"
  if helm template "$(release_for "$env")" "$CHART_DIR" -f "$values" -n "$(namespace_for "$env")" > "$out"; then
```

```
      grep -q "APP_ENV: \"$(app_env_for "$env")\"" "$out" \
        && echo "APP_ENV correctly rendered as $(app_env_for "$env")" \
        || { echo "APP_ENV mismatch in $out"; failures=$((failures + 1)); }
    else
      failures=$((failures + 1))
    fi

    if command -v kubeconform >/dev/null 2>&1; then
      log "[$env] kubeconform"
      kubeconform -strict -summary -kubernetes-version "${KUBERNETES_VERSION:-1.31.0}" "$out" \
        || failures=$((failures + 1))
    else
      echo "kubeconform not installed - skipping Kubernetes schema validation"
    fi
  done

  log "Rendered manifests written to ${OUT_DIR}"
  if [[ $failures -gt 0 ]]; then
    echo "VALIDATION FAILED (${failures} problem(s))" >&2
    exit 1
  fi
  echo "All environments validated successfully."
```

Code Snippet – scripts/test-environments.sh (complete, 68 lines)

```
  #!/usr/bin/env bash
  # Live verification of deployed environments. For each environment it checks:
  #   - the Helm release is deployed
  #   - all desired replicas are ready
  #   - the running app reports the expected environment (via /api/config)
  #   - /health returns ok
  #   - `helm test` passes
  #
  # Usage: ./scripts/test-environments.sh [dev|test|staging|prod|all]   (default: all)
  set -uo pipefail
  source "$(dirname "$0")/common.sh"
  require_cmd kubectl helm curl python3

  TARGET="${1:-all}"
  [[ "$TARGET" == "all" ]] && ENVS=("${ALL_ENVS[@]}") || { require_env "$TARGET"; ENVS=("$TARGET"); }

  pass=0; fail=0
  check() {  # check <description> <command...>
    local desc="$1"; shift
    if "$@"; then echo "  PASS  $desc"; pass=$((pass + 1)); else echo "  FAIL  $desc"; fail=$((fail +
     1)); fi
  }

  json_field() {  # json_field <python expression on d>
    python3 -c "import json,sys; d=json.load(sys.stdin); print($1)"
  }

  local_port=18080
  for env in "${ENVS[@]}"; do
    ns="$(namespace_for "$env")"; release="$(release_for "$env")"; expected="$(app_env_for "$env")"
    log "Testing environment '${env}' (release ${release}, namespace ${ns})"

    if ! helm status "$release" -n "$ns" >/dev/null 2>&1; then
      echo "  FAIL  release ${release} is not installed"; fail=$((fail + 1)); continue
    fi
    check "helm release status is deployed" \
      bash -c "helm status '$release' -n '$ns' -o json | python3 -c 'import json,sys;
     sys.exit(json.load(sys.stdin)[\"info\"][\"status\"] != \"deployed\")'"

    deploy="$(kubectl get deploy -n "$ns" -l "app.kubernetes.io/instance=${release}" -o
     jsonpath='{.items[0].metadata.name}')"
    desired="$(kubectl get deploy "$deploy" -n "$ns" -o jsonpath='{.spec.replicas}')"
    ready="$(kubectl get deploy "$deploy" -n "$ns" -o jsonpath='{.status.readyReplicas}')"
    check "replicas ready (${ready:-0}/${desired})" test "${ready:-0}" = "$desired"

    svc="$(kubectl get svc -n "$ns" -l "app.kubernetes.io/instance=${release}" -o
     jsonpath='{.items[0].metadata.name}')"
    kubectl port-forward -n "$ns" "svc/${svc}" "${local_port}:80" >/dev/null 2>&1 &
    pf_pid=$!
    for _ in $(seq 1 20); do curl -fs "http://127.0.0.1:${local_port}/health" >/dev/null 2>&1 && break;
     sleep 0.5; done

    body="$(curl -fs "http://127.0.0.1:${local_port}/api/config" || echo '{}')"
    actual="$(echo "$body" | json_field 'd.get("environment")')"
    check "APP_ENV is '${expected}' (got '${actual}')" test "$actual" = "$expected"
    replicas_cfg="$(echo "$body" | json_field 'd.get("settings",{}).get("REPLICA_COUNT")')"
    check "REPLICA_COUNT in ConfigMap matches deployment (${replicas_cfg})" test "$replicas_cfg" =
     "$desired"
    check "/health returns ok" bash -c "curl -fs http://127.0.0.1:${local_port}/health | grep -q ok"
    if [[ "$env" == "prod" ]]; then
      debug="$(echo "$body" | json_field 'd["settings"]["DEBUG_MODE"]')"
      check "production DEBUG_MODE is False" test "$debug" = "False"
      src="$(echo "$body" | json_field 'd["settings"]["SECRET_KEY"]')"
```

```
    check "production SECRET_KEY comes from a Kubernetes Secret" bash -c "echo '$src' | grep -q kubernetes-secret"
  fi
  kill "$pf_pid" >/dev/null 2>&1; wait "$pf_pid" 2>/dev/null
  local_port=$((local_port + 1))

  check "helm test ${release}" bash -c "helm test '$release' -n '$ns' --timeout 2m >/dev/null"
done

echo
echo "Result: ${pass} passed, ${fail} failed"
[[ $fail -eq 0 ]]
```

Code Snippet – scripts/upgrade-rollback-demo.sh (complete, 50 lines)

```
#!/usr/bin/env bash
# Demonstrates the Helm release lifecycle on one environment:
#   install (v1.0.0) -> upgrade (v1.1.0) -> history -> rollback -> history
#
# Usage: ./scripts/upgrade-rollback-demo.sh [dev|test|staging|prod]   (default: dev)
set -euo pipefail
source "$(dirname "$0")/common.sh"
require_cmd kubectl helm

ENV_NAME="${1:-dev}"
require_env "$ENV_NAME"
NS="$(namespace_for "$ENV_NAME")"
RELEASE="$(release_for "$ENV_NAME")"
VALUES="$(values_for "$ENV_NAME")"
NEW_VERSION="${NEW_VERSION:-1.1.0}"

show_version() {
  kubectl get configmap -n "$NS" -l "app.kubernetes.io/instance=${RELEASE}" \
    -o jsonpath='{.items[0].data.APP_VERSION}'
}

if ! helm status "$RELEASE" -n "$NS" >/dev/null 2>&1; then
  log "Release not found - installing first"
  "$(dirname "$0")/deploy.sh" "$ENV_NAME"
fi

ORIGINAL="$(show_version)"
log "Current version: ${ORIGINAL}"

log "Upgrading ${RELEASE} to APP_VERSION ${NEW_VERSION} and LOG_LEVEL=INFO"
# --reuse-values is NOT used on purpose: the values file is always the source of truth.
helm upgrade "$RELEASE" "$CHART_DIR" -n "$NS" -f "$VALUES" \
  --set app.version="${NEW_VERSION}" --set config.LOG_LEVEL=INFO \
  --atomic --wait --timeout 5m
UPGRADED="$(show_version)"
echo "Version after upgrade: ${UPGRADED}"
[[ "$UPGRADED" == "$NEW_VERSION" ]] || { echo "ERROR: upgrade did not apply" >&2; exit 1; }

log "Release history"
helm history "$RELEASE" -n "$NS"

PREVIOUS="$(helm history "$RELEASE" -n "$NS" -o json | python3 -c 'import json,sys; h=json.load(sys.stdin); print(h[-2]["revision"])')"
log "Rolling back to revision ${PREVIOUS}"
helm rollback "$RELEASE" "$PREVIOUS" -n "$NS" --wait --timeout 5m
RESTORED="$(show_version)"
echo "Version after rollback: ${RESTORED}"
[[ "$RESTORED" == "$ORIGINAL" ]] || { echo "ERROR: rollback did not restore ${ORIGINAL}" >&2; exit 1; }

log "Release history after rollback"
helm history "$RELEASE" -n "$NS"
```

Appendix F – GitHub Actions validation workflow

Code Snippet – .github/workflows/helm-validation.yml (complete, 117 lines)

```
name: Helm Chart Validation

on:
  push:
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

env:
  HELM_VERSION: v3.16.2
  KUBECONFORM_VERSION: v0.6.7
```

```
  KUBERNETES_VERSION: 1.31.0
  IMAGE: multi-env-app:1.0.0

jobs:
  python-tests:
    name: Unit & configuration tests (pytest)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - uses: actions/setup-python@v6
        with:
          python-version: "3.12"

      - uses: azure/setup-helm@v4
        with:
          version: ${{ env.HELM_VERSION }}

      - name: Install dependencies
        run: pip install -r requirements-dev.txt

      - name: Run pytest (app + per-environment Helm config tests)
        run: python -m pytest -v

  helm-validate:
    name: Helm lint / template / kubeconform
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - uses: azure/setup-helm@v4
        with:
          version: ${{ env.HELM_VERSION }}

      - name: Install kubeconform
        run: |
          curl -sSL "https://github.com/yannh/kubeconform/releases/download/${KUBECONFORM_VERSION}/kubeconform-linux-amd64.tar.gz" \
            | tar -xz -C /usr/local/bin kubeconform
          kubeconform -v

      - name: Validate chart for dev, test, staging and prod
        run: ./scripts/validate-helm.sh

      - name: Upload rendered manifests
        uses: actions/upload-artifact@v4
        with:
          name: rendered-manifests
          path: dist/rendered/

  docker-build:
    name: Docker build & smoke test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - name: Build image
        run: docker build -t "$IMAGE" .

      - name: Run container and check endpoints
        run: |
          docker run -d --name smoke -p 5000:5000 -e APP_ENV=testing "$IMAGE"
          for i in $(seq 1 20); do curl -fs http://127.0.0.1:5000/health && break; sleep 1; done
          curl -fs http://127.0.0.1:5000/api/config | tee /tmp/config.json
          python3 -c "import json; d=json.load(open('/tmp/config.json')); assert d['environment']=='testing', d"
          curl -fs http://127.0.0.1:5000/ | grep -q 'id="app-env"'
          docker rm -f smoke

  deploy-kind:
    name: Deploy all environments to a kind cluster
    runs-on: ubuntu-latest
    needs: [python-tests, helm-validate, docker-build]
    steps:
      - uses: actions/checkout@v5

      - uses: azure/setup-helm@v4
        with:
          version: ${{ env.HELM_VERSION }}

      - name: Create kind cluster
        uses: helm/kind-action@v1
        with:
          cluster_name: chart-testing

      - name: Build image and load it into kind
        run: |
          docker build -t "$IMAGE" .
          kind load docker-image "$IMAGE" --name chart-testing
          docker pull busybox:1.36
          kind load docker-image busybox:1.36 --name chart-testing
```

```
103
104         - name: Deploy dev, test, staging and prod
105           run: ./scripts/deploy.sh all
106
107         - name: Verify environment-specific configuration (live)
108           run: ./scripts/test-environments.sh all
109
110         - name: Upgrade and rollback (dev)
111           run: ./scripts/upgrade-rollback-demo.sh dev
112
113         - name: Show cluster state
114           if: always()
115           run: |
116             helm list -A
117             kubectl get pods -A -l app.kubernetes.io/name=multi-env-app -o wide
```

Appendix G – Other files and reproduction of the results

Table A.1 lists the files of the repository that are not printed above; they are part of the repository and of the delivered archive. The evidence files of Chapter IV are in `docs/evidence/` (outputs of the cluster run `00-*` to `10-*`, the availability results `08-availability.csv`, and the screenshots and `/api/config` answers `dashboard-*`).

Table A.1 – Files of the repository that are not printed

File	Lines	Purpose
`.github/workflows/capture-evidence.yml`	56	Workflow that records the evidence on kind and commits it
`README.md`	447	Documentation: setup on Minikube and Windows, commands, troubleshooting
`helm/multi-env-app/.helmignore`	10	Files excluded from the chart package
`helm/multi-env-app/templates/NOTES.txt`	17	Notes printed after an install
`helm/multi-env-app/values-staging.yaml`	28	Values of the staging environment
`helm/multi-env-app/values-test.yaml`	28	Values of the testing environment
`requirements-dev.txt`	3	Test dependencies (pytest, PyYAML)
`scripts/capture-evidence.sh`	238	Records the evidence of Chapter IV on a cluster
`scripts/cleanup.sh`	12	Uninstalls the releases and deletes the namespaces
`scripts/docker-demo.sh`	63	Runs the four environments as plain Docker containers
`scripts/minikube-setup.sh`	25	Starts Minikube, builds the image inside it, creates the namespaces
`tests/conftest.py`	9	Adds the application folder to the import path
`tests/test_app.py`	93	15 application tests
`tests/test_helm_config.py`	230	32 static and 15 Helm-based tests of the values and the chart

The commands below reproduce the results on a machine with Python, Docker, Helm, kubectl and either Minikube or kind. The first two need no cluster.

Code Snippet – Reproduction of the results

```
pip install -r requirements-dev.txt
python -m pytest -q                          # 62 cases (some need helm)
./scripts/validate-helm.sh                   # lint, render, kubeconform

./scripts/minikube-setup.sh                  # or: a kind cluster + image
./scripts/deploy.sh all                      # four environments
./scripts/test-environments.sh all           # 26 live checks
./scripts/upgrade-rollback-demo.sh dev       # upgrade, history, rollback
./scripts/capture-evidence.sh                # evidence (needs Chrome)
./scripts/cleanup.sh
```

REFERENCES

[1] A. Wiggins, “III. Config,” *The Twelve-Factor App*. https://12factor.net/config [Accessed: 20 Oct. 2026.]

[2] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy, “An empirical study on configuration errors in commercial and open source systems,” in *Proc. 23rd ACM Symp. Operating Systems Principles (SOSP '11)*, Cascais, Portugal, 2011, pp. 159–172, doi: 10.1145/2043556.2043572.

[3] T. Xu and Y. Zhou, “Systems approaches to tackling configuration errors: A survey,” *ACM Comput. Surv.*, vol. 47, no. 4, Art. 70, Jul. 2015, doi: 10.1145/2791577.

[4] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ, USA: Addison-Wesley, 2010.

[5] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. Portland, OR, USA: IT Revolution Press, 2018.

[6] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” *Linux J.*, vol. 2014, no. 239, Mar. 2014.

[7] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in *Proc. 10th European Conf. Computer Systems (EuroSys '15)*, Bordeaux, France, 2015.

[8] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016. https://queue.acm.org/detail.cfm?id=2898444

[9] The Kubernetes Authors, “Deployments,” *Kubernetes Documentation*. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/ [Accessed: 20 Oct. 2026.]

[10] The Helm Authors, “Charts,” *Helm Documentation*. https://helm.sh/docs/topics/charts/ [Accessed: 20 Oct. 2026.]

[11] The Helm Authors, “Values files,” *Helm Documentation*. https://helm.sh/docs/chart_template_guide/values_files/ [Accessed: 20 Oct. 2026.]

[12] The Helm Authors, “helm rollback,” *Helm Documentation*. https://helm.sh/docs/helm/helm_rollback/ [Accessed: 20 Oct. 2026.]

[13] A. Zerouali, R. Opdebeeck, and C. De Roover, “Helm charts for Kubernetes applications: Evolution, outdatedness and security risks,” in *Proc. 20th IEEE/ACM Int. Conf. Mining Software Repositories (MSR)*, Melbourne, Australia, 2023.

[14] A. Rahman and L. Williams, “Characterizing defective configuration scripts used for continuous deployment,” in *Proc. 11th IEEE Int. Conf. Software Testing, Verification and Validation (ICST)*, 2018.

[15] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, “A systematic mapping study of infrastructure as code research,” *Inf. Softw. Technol.*, vol. 108, pp. 65–77, 2019.

[16] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, “Security misconfigurations in open source Kubernetes manifests: An empirical study,” *ACM Trans. Softw. Eng. Methodol.*, 2023.

[17] The Kubernetes Authors, “kind,” Kubernetes SIGs. https://kind.sigs.k8s.io/ [Accessed: 20 Oct. 2026.]

[18] The Kubernetes Authors, “minikube start,” *Minikube Documentation*. https://minikube.sigs.k8s.io/docs/start/ [Accessed: 20 Oct. 2026.]

[19] T. A. Limoncelli, “GitOps: A path to more self-service IT,” *ACM Queue*, vol. 16, no. 3, pp. 13–26, 2018, doi: 10.1145/3236386.3237207.

[20] The Helm Authors, “Chart tests,” *Helm Documentation*. https://helm.sh/docs/topics/chart_tests/ [Accessed: 20 Oct. 2026.]

[21] The Helm Authors, “helm lint,” *Helm Documentation*. https://helm.sh/docs/helm/helm_lint/ [Accessed: 20 Oct. 2026.]

[22] The Helm Authors, “helm upgrade,” *Helm Documentation*. https://helm.sh/docs/helm/helm_upgrade/ [Accessed: 20 Oct. 2026.]

[23] The Kubernetes Authors, “ConfigMaps,” *Kubernetes Documentation*. https://kubernetes.io/docs/concepts/configuration/configmap/ [Accessed: 20 Oct. 2026.]

[24] The Kubernetes Authors, “Secrets,” *Kubernetes Documentation*. https://kubernetes.io/docs/concepts/configuration/secret/ [Accessed: 20 Oct. 2026.]

[25] The Kubernetes Authors, “Configure liveness, readiness and startup probes,” *Kubernetes Documentation*. https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/ [Accessed: 20 Oct. 2026.]

[26] The Kubernetes Authors, “Service,” *Kubernetes Documentation*. https://kubernetes.io/docs/concepts/services-networking/service/ [Accessed: 20 Oct. 2026.]

[27] GitHub, Inc., “Workflow syntax for GitHub Actions,” *GitHub Docs*. https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions [Accessed: 20 Oct. 2026.]

[28] The Kubernetes Authors, “Declarative management of Kubernetes objects using Kustomize,” *Kubernetes Documentation*. https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/ [Accessed: 20 Oct. 2026.]

[29] Pallets Projects, “Flask Documentation.” https://flask.palletsprojects.com/ [Accessed: 20 Oct. 2026.]

[30] The Kubernetes Authors, “Expose Pod information to containers through environment variables,” *Kubernetes Documentation*. https://kubernetes.io/docs/tasks/inject-data-application/environment-variable-expose-pod-information/ [Accessed: 20 Oct. 2026.]

[31] Bootstrap contributors, “Bootstrap 5.3 Documentation.” https://getbootstrap.com/docs/5.3/ [Accessed: 20 Oct. 2026.]

[32] Docker Inc., “Dockerfile reference,” *Docker Documentation*. https://docs.docker.com/reference/dockerfile/ [Accessed: 20 Oct. 2026.]

[33] B. Chesneau and contributors, “Gunicorn – Python WSGI HTTP Server for UNIX.” https://docs.gunicorn.org/ [Accessed: 20 Oct. 2026.]

[34] The Helm Authors, “Values” (Chart Best Practices), *Helm Documentation*. https://helm.sh/docs/chart_best_practices/values/ [Accessed: 20 Oct. 2026.]

[35] JSON Schema organization, “JSON Schema.” https://json-schema.org/ [Accessed: 20 Oct. 2026.]

[36] The Kubernetes Authors, “Namespaces,” *Kubernetes Documentation*. https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/ [Accessed: 20 Oct. 2026.]

[37] pytest-dev, “pytest documentation.” https://docs.pytest.org/ [Accessed: 20 Oct. 2026.]

[38] The kubeconform contributors, “kubeconform,” GitHub repository. https://github.com/yannh/kubeconform [Accessed: 20 Oct. 2026.]

[39] The Kubernetes Authors, “Pod lifecycle: termination of Pods,” *Kubernetes Documentation*. https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#pod-termination [Accessed: 20 Oct. 2026.]

[40] The Helm Authors, “pkg/kube/ready.go,” Helm source code, tag v3.16.2. https://github.com/helm/helm/blob/v3.16.2/pkg/kube/ready.go [Accessed: 20 Oct. 2026.]